



โครงการวิศวกรรมคอมพิวเตอร์และปัญญาประดิษฐ์ปีการศึกษา 2568

ระบบซื้อขายพลังงานแสงอาทิตย์แบบ Peer-to-Peer ผ่าน Smart Contract บนเครือข่าย Solana-compatible Consortium

นาย จันทรวัฒน์ กิริยาดี 2410717302003

อาจารย์ที่ปรึกษาโครงการ : _____

สาขาวิชาวิศวกรรมคอมพิวเตอร์และปัญญาประดิษฐ์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยหอการค้าไทย

บทคัดย่อ : The growth of rooftop solar generation turns electricity consumers into prosumers who hold surplus energy and wish to trade it directly, but Peer-to-Peer (P2P) trading remains hard because of grid constraints and the absence of a transparent pricing mechanism. This project presents the design and architectural evaluation of a P2P energy-trading system built on a permissioned Solana-compatible consortium blockchain that is governable and has predictable transaction cost. The core of the design is a clean separation between data verification and settlement: off-chain verification is performed by an Aggregator Bridge that checks the Ed25519 signature of every meter reading before orders are matched by a Continuous Double Auction (CDA), while on-chain settlement records only verified matches and enforces its correctness conditions in a single atomic transaction. The contribution therefore lies not in any single component but in the integration of all of them behind an explicitly defined trust boundary. The evaluation on the simulated system supports this design: the ingest path sustains 80 meters at the nominal design rate of 5.33 readings per second with no data loss, and a stepped load ramp up to 640 meters keeps the loss ratio at or below 0.03%. The CDA matching engine processes about 3.1×10^4 matched pairs per second in memory, on-chain settlement costs 96,707 compute units per matched pair (about 48% of the default budget), and the real settlement path measures about 0.5 transactions per second on a single validator — global-write-bound by design, yet still supporting roughly 450 settlements per 900-second market-clearing window, far beyond the tested workload. These results support the use of the blockchain as a thin settlement layer. This is an architectural evaluation on a simulated system, not a measurement on a live power network.

1. บทนำ

การขยายตัวของพลังงานหมุนเวียนทำให้ผู้ใช้ไฟฟ้าจำนวนมากเปลี่ยนบทบาทจากผู้บริโภคเพียงอย่างเดียวไปสู่การเป็นผู้ผลิตและผู้บริโภคพลังงานในเวลาเดียวกัน (Prosumer) การเปลี่ยนแปลงนี้สร้างความท้าทายต่อโครงข่ายไฟฟ้าแบบดั้งเดิม ซึ่งออก

แบบมาสำหรับการจ่ายไฟจากศูนย์กลางไปยังผู้ใช้ปลายทางเป็นหลัก [1] การเปลี่ยนผ่านสู่ระบบ Smart Grid ในบริบทประเทศไทยมีการใช้โครงสร้างพื้นฐานมาตรวัดขั้นสูง (Advanced Metering Infrastructure: AMI) ทำให้ข้อมูลการผลิตและการใช้พลังงานมีความละเอียดสูงขึ้น การบูรณาการทรัพยากรพลังงานแบบกระจายศูนย์ (Distributed Energy Resources: DER) เช่น ระบบผลิตไฟฟ้าพลังงานแสงอาทิตย์บนหลังคาและระบบกักเก็บพลังงานด้วยแบตเตอรี่ จึงต้องคำนึงข้อกำหนดด้านการเชื่อมต่อ DER, Microgrid controller และข้อกำหนดของโครงข่ายแรงดันต่ำ [2] [22] [23]

ปัญหาหลักของการซื้อขายไฟฟ้าระหว่างผู้ใช้โดยตรงมีสองด้าน ด้านแรกคือกลไกตั้งราคาที่ไม่โปร่งใสและตรวจสอบย้อนหลังได้ ด้านที่สองคือการพิสูจน์ว่าปริมาณไฟฟ้าที่ซื้อขายนั้นมาจากการผลิตจริง งานวิจัยจำนวนมากเสนอให้ใช้บล็อกเชนเป็นชั้นบันทึกธุรกรรม แต่ส่วนใหญ่ประเมินบนเครือข่ายสาธารณะที่มีต้นทุนธุรกรรมผันผวนและความหน่วงสูง [3] ขณะที่การนำไปใช้ในภาคพลังงานต้องการต้นทุนที่คาดการณ์ได้และการกำกับดูแลที่ควบคุมได้ [4]

โครงการนี้จะออกแบบและพัฒนาระบบจำลองการซื้อขายพลังงานแบบ Peer-to-Peer บนเครือข่าย consortium blockchain แบบ permissioned ที่กำกับการเข้าร่วมแบบ Proof of Authority (PoA) โดยมีส่วนสนับสนุนหลักสามประการ ประการแรก การออกแบบสถาปัตยกรรมแบบลดการพึ่งพิงกันที่แยกการตรวจสอบข้อมูลนอกเชนผ่าน Aggregator Bridge (การตรวจลายเซ็น Ed25519 ของข้อมูลมาตรวัดตามมาตรฐาน DLMS/COSEM และการประเมินเงื่อนไข Grid stability) ร่วมกับการจับคู่คำสั่งด้วย Continuous Double Auction (CDA) ออกจากการชำระธุรกรรมบนเชนอย่างชัดเจน ประการที่สอง การออกแบบขอบเขตความเชื่อถือของการชำระธุรกรรมบนเครือข่าย Anchor/Solana-compatible โดยบล็อกเชนตรวจสอบลายเซ็น Ed25519 ของทั้งผู้ซื้อและผู้ขาย การกันส่งข้ามผ่าน order nullifier และสถานะ escrow ขณะที่เงื่อนไขปริมาณพลังงานและ oracle attestation ถูกบังคับใช้ในชั้น off-chain ทำให้บล็อกเชนทำหน้าที่เป็นชั้น settlement และ audit อย่างชัดเจน และประการที่สาม การพัฒนาชุดจำลอง AMI ที่อาศัยแบบจำลองโครงข่ายด้วย pandapower และ Smart Meter Simulator เพื่อสร้างข้อมูลมาตรวัดแบบ deterministic พร้อมการวัดคุณลักษณะของระบบแบบทำซ้ำได้

จุดต่างหลักจากงานก่อนหน้าไม่ได้อยู่ที่องค์ประกอบเดี่ยว (consortium blockchain, CDA หรือ off-chain oracle ซึ่งล้วนมีในงานก่อนหน้า) แต่อยู่ที่การผสานองค์ประกอบเหล่านี้เข้าด้วยกันผ่านขอบเขตความเชื่อถือที่นิยามชัด คือ CDA แบบแบ่งโซนบนเครือข่าย consortium PoA ที่แยกชั้นตรวจสอบ telemetry (Ed25519/DLMS) นอกเชนออกจากชั้น settlement อย่างชัดเจน โดยกลไกการชำระบนเชนที่เป็นแกนของงานนี้คือการส่งมอบพร้อมชำระแบบ atomic (DvP) ที่รวมการตรวจลายเซ็น Ed25519 ของทั้งสองฝ่าย การกันส่งข้ามแบบ partial-fill ผ่าน order nullifier และเพดานการไหลข้ามโซนไว้เป็นชุดเงื่อนไขความถูกต้องเดียวที่บังคับใช้ในธุรกรรมเดียว ซึ่งงานก่อนหน้ามิได้ระบุไว้อย่างเป็นระบบ ทั้งนี้ขอบเขตของงานเป็นการออกแบบและประเมินเชิงสถาปัตยกรรมบนระบบจำลอง ไม่ใช่การวัดจากเครือข่ายไฟฟ้าภาคสนามหรือเครือข่าย Solana production ส่วนสนับสนุนเชิงประจักษ์ของงานจึงเป็นการบ่งชี้คุณลักษณะ (characterization) ของระบบเดี่ยวแบบทำซ้ำได้ ได้แก่ อัตราการรับข้อมูล ต้นทุน compute-unit และปริมาณงานของเส้นทางชำระ มิใช่การเปรียบเทียบเชิงปริมาณแบบ head-to-head กับแพลตฟอร์มอื่น ซึ่งเป็นงานในอนาคต

บทความนี้จัดเรียงเนื้อหา ดังนี้ หัวข้อ 2 ทบทวนทฤษฎี งานวิจัยที่เกี่ยวข้อง และเครื่องมือที่ใช้ หัวข้อ 3 กำหนดแบบจำลองระบบความเชื่อถือ และผู้โจมตี หัวข้อ 4 อธิบายการออกแบบระบบทั้งหมด ตั้งแต่สถาปัตยกรรม ชุดจำลอง เครือข่าย consortium กลไกราคา โปรแกรมบนเชน จนถึงวงจรการซื้อขาย หัวข้อ 5 รายงานผลการทดลอง หัวข้อ 6 อภิปรายผลและข้อจำกัด และ หัวข้อ 7 สรุปผลการศึกษาและข้อเสนอแนะ ตัวอย่างที่ใช้อยู่ในบทความสรุปไว้ในตารางที่ 1

ตัวย่อ	ความหมาย	ตัวย่อ	ความหมาย
AMI	Advanced Metering Infrastructure (โครงสร้างพื้นฐานมาตรวัดขั้นสูง)	DLMS/COSEM	Device Language Message Specification / COSEM (IEC 62056)
BESS	Battery Energy Storage System	DSO	Distribution System Operator
BFT	Byzantine Fault Tolerance	DvP	Delivery-versus-Payment (ส่งมอบพร้อมชำระเงินแบบ atomic)
CDA	Continuous Double Auction (ตลาดประมูลสองทางแบบต่อเนื่อง)	GRX / THBG	โทเคนกำกับดูแล / stablecoin ดริงเงินบาท
CPI	Cross-Program Invocation	IAM	Identity and Access Management
CU	Compute Unit (หน่วยประมวลผลบนเชนของ Solana)	mTLS	Mutual TLS

DAO	Decentralized Autonomous Organization	OPF	Optimal Power Flow
DER	Distributed Energy Resource (ทรัพยากรพลังงานแบบกระจายศูนย์)	PDA	Program Derived Address
PoA	Proof of Authority	SPIFFE	Secure Production Identity Framework for Everyone
PoH	Proof of History	SPL	Solana Program Library (Token)
RBAC	Role-Based Access Control	SVM	Solana Virtual Machine
REC	Renewable Energy Certificate	VPP	Virtual Power Plant

ตารางที่ 1: ตัวย่อที่ใช้ในบทความ

2. ทฤษฎีและงานวิจัยที่เกี่ยวข้องและเครื่องมือที่ใช้ในการออกแบบ

2.1 บล็อกเชนและ Smart Contract

เทคโนโลยีบล็อกเชนถือกำเนิดจาก Bitcoin [5] ในฐานะระบบบัญชีแยกประเภทแบบกระจายศูนย์ที่ไม่ต้องอาศัยตัวกลาง และขยายขีดความสามารถสู่การประมวลผล Smart Contract ด้วย Ethereum [6] [25] ซึ่งทำให้สามารถเขียนตรรกะทางธุรกิจให้ทำงานบนเซนได้โดยตรง ภาพรวมของเทคโนโลยีและการจำแนกประเภทเครือข่ายแบบ permissionless และ permissioned ได้รับการสรุปไว้โดย NIST [7] ซึ่งเป็นกรอบอ้างอิงในการเลือกสถาปัตยกรรมเครือข่ายให้เหมาะสมกับข้อกำหนดด้านการกำกับดูแล โดยเครือข่ายแบบ permissioned เปิดให้กำหนดสิทธิ์ผู้เข้าร่วมและนโยบายการอัปเดตได้ ซึ่งเป็นคุณสมบัติที่จำเป็นสำหรับระบบไฟฟ้าที่ต้องอยู่ภายใต้การกำกับดูแล

งานนี้ใช้เครือข่าย Solana-compatible ซึ่งประมวลผลโปรแกรมบน Solana Virtual Machine (SVM) [31] โปรแกรมบนเซนใช้บัญชีแบบ Program Derived Address (PDA) [32] ซึ่งเป็นที่อยู่บัญชีที่คำนวณได้แบบ deterministic จาก seed และรหัสโปรแกรม ทำให้โปรแกรมตรวจสอบความเป็นเจ้าของบัญชีได้โดยไม่ต้องถือกุญแจส่วนตัวของบัญชีสถานะแต่ละรายการ ส่วนโทเคนใช้มาตรฐาน SPL Token และ Token-2022 [33] สำหรับการสร้าง โอน และเผาโทเคน

2.2 กลไกฉันทามติ

ปัญหา Byzantine Generals [26] เป็นรากฐานเชิงทฤษฎีของการบรรลุข้อตกลงในระบบกระจายที่มีโหนดประพฤติผิดกติก และอัลกอริทึม Practical Byzantine Fault Tolerance (PBFT) [9] แสดงให้เห็นว่าการยืนยันธุรกรรมทำได้จริงในเครือข่ายที่มีโหนดจำนวนจำกัด ซึ่งสอดคล้องกับสมมติฐานเครือข่าย permissioned ที่ควบคุมการเข้าร่วม validator แบบ Proof of Authority (PoA) [27] ในงานนี้

เครือข่าย Solana-compatible แยกกลไกฉันทามติออกเป็นสองส่วนที่ทำงานร่วมกัน ได้แก่ Proof of History (PoH) ที่เป็นนาฬิกาเชิงรหัสวิทยา (verifiable clock) สำหรับเรียงลำดับเหตุการณ์ตามเวลาก่อนการลงมติ และ Tower BFT ซึ่งเป็นกลไกลงมติแบบ Byzantine Fault Tolerant ที่พัฒนาต่อยอดจากแนวคิด PBFT สำหรับยืนยันบล็อกและประกาศสิ้นสุดบล็อก (finality) [8] สิ่งสำคัญที่ต้องเน้นย้ำคือ PoA ในงานนี้เป็นขั้นกักกับการเข้าร่วม (admission control) มิใช่การแทนที่กลไกฉันทามติระดับ Layer 1

2.3 ตลาดพลังงานแบบ Peer-to-Peer และงานวิจัยที่เกี่ยวข้อง

ในบริบทของตลาดพลังงานแบบ Peer-to-Peer มีงานวิจัยจำนวนมากที่ศึกษาตลาดและการออกแบบการประมูล Mengelkamp และคณะ [10] เปรียบเทียบการออกแบบตลาดพลังงานท้องถิ่นและกลยุทธ์การเสนอราคา Munsing และคณะ [11] เสนอการใช้บล็อกเชนเพื่อกระจายการหาค่าเหมาะที่สุด (decentralized optimization) ของทรัพยากรพลังงานในเครือข่ายไมโครกริด ขณะที่ Morstyn และคณะ [20] เสนอเครือข่ายสัญญาทวิภาคี (bilateral contract network) และ Paudel และคณะ [21] วิเคราะห์การซื้อขายในชุมชนไมโครกริดด้วยแบบจำลองเชิงทฤษฎีเกม งานเหล่านี้ชี้ให้เห็นถึงศักยภาพของบล็อกเชนในการรองรับการซื้อขายพลังงานโดยตรงระหว่างผู้ใช้ แต่ส่วนใหญ่ยังประเมินบนเครือข่ายสาธารณะที่มีข้อจำกัดด้านต้นทุนธุรกรรมและความหน่วงในการยืนยันบล็อก ตัวอย่างที่เป็นรูปธรรมล่าสุดคือแพลตฟอร์มซื้อขายแบบกระจายศูนย์ของ Esmat และคณะ [3] ที่พัฒนากลไกตลาดและการชำระบนบล็อกเชนสาธารณะ (Ethereum) แต่ไม่ได้แยกชั้นการตรวจสอบข้อมูลนอกเซนจากการชำระบนเซนอย่างชัดเจน

เพื่อตอบโจทย์ด้านการกำกับดูแลและความสามารถในการคาดการณ์ต้นทุน งานวิจัยจำนวนหนึ่งจึงหันมาใช้เครือข่ายแบบ Consortium หรือ permissioned โดย Kang และคณะ [12] ใช้ consortium blockchain สำหรับการซื้อขายไฟฟ้าแบบ Peer-to-Peer ระหว่างยานยนต์ไฟฟ้าแบบ plug-in และ Hyperledger Fabric [13] เป็นตัวอย่างของระบบปฏิบัติการแบบกระจายสำหรับเครือข่าย permissioned ที่กำหนดสิทธิ์ผู้เข้าร่วมได้ชัดเจน

การทบทวนเชิงระบบของ Andoni และคณะ [4] สํารวจการประยุกต์บล็อกเชนในภาคพลังงานอย่างครอบคลุมและชี้ความท้าทายหลักด้านความสามารถในการขยายขนาด ต้นทุน และการกำกับดูแล งานทบทวนวรรณกรรมล่าสุดยังสะท้อนว่าหัวข้อนี้ยังเป็นพื้นที่วิจัยที่เปิดอยู่ โดย Tanis และคณะ [28] ทบทวนโครงสร้างตลาด ขั้นตอนการดำเนินงาน และระบบหลายพลังงานของการซื้อขายแบบ Peer-to-Peer Bhavana และคณะ [29] สํารวจการประยุกต์บล็อกเชนในตลาดพลังงานและห่วงโซ่อุปทานไฮโดรเจนสีเขียว และการประเมินเปรียบเทียบกลไกฉันทามติสำหรับการซื้อขายพลังงานในไมโครกริด [30] สนับสนุนการเลือกเครือข่าย permissioned ที่ควบคุมสิทธิ์ได้

ต่างจากงานข้างต้น บทความนี้มุ่งเน้นการออกแบบและประเมินสถาปัตยกรรมของระบบจำลองที่แยกการตรวจสอบนอกเชน (off-chain verification) ผ่าน Aggregator Bridge ออกจากขั้น Settlement บนบล็อกเชนอย่างชัดเจน ซึ่งสอดคล้องกับแนวทางเบื้องต้นในการทดสอบระบบควบคุมไมโครกริด [24] ตำแหน่งของงานนี้เทียบกับงานก่อนหน้าสรุปไว้ในตารางที่ 2 ทั้งนี้ ตารางที่ 2 เป็นการเทียบเชิงคุณภาพ (qualitative positioning) เนื่องจากงานเหล่านี้รับบนเครือข่าย ชุดข้อมูล และสมมติฐานที่ต่างกัน จึงไม่มีตัวชี้วัดเชิงปริมาณร่วมที่เทียบกันได้โดยตรง

2.4 เครื่องมือที่ใช้ในการออกแบบ

บริการฝั่ง Backend ทุกตัวพัฒนาด้วยภาษา Rust (Axum) บน Tokio async runtime สื่อสารระหว่างกันผ่าน ConnectRPC บน mutually authenticated TLS (mTLS) ที่ผูกตัวตนของบริการด้วย SPIFFE X.509 identity [34] และแยกเส้นทางเขียนข้อมูลลงบล็อกเชนผ่าน NATS JetStream [14] ซึ่งเป็นระบบคิวข้อความแบบ persistent ที่รองรับการส่งซ้ำและการรับประกันการส่งมอบ โปรแกรมบนเซิร์ฟเวอร์พัฒนาด้วยเฟรมเวิร์ก Anchor [15] ซึ่งกำหนด account validation, instruction handler และ Event log อย่างเป็นระบบ และใช้ Program Derived Address (PDA) [32] เพื่อสร้างบัญชีที่ตรวจสอบ ownership ได้แบบ deterministic

ส่วนชุดจำลองโครงข่ายและ Smart Meter พัฒนาด้วย Python 3.11 [35] โดยใช้ FastAPI [36] สำหรับ REST endpoint, NetworkX [16] แทนโครงสร้าง topology, pvlb [17] จำลองกำลังผลิตจากแผงเซลล์แสงอาทิตย์ และ pandapower [18] คำนวณ power flow โดยรับโครงสร้างโครงข่ายจากไฟล์ GridLAB-D GLM [19] การเลือกใช้ pandapower ทำให้ระบบสร้างข้อมูลมาตรวัดที่ผ่านการตรวจสอบทางฟิสิกส์ (physics-validated) แบบ deterministic สำหรับโครงข่ายไฟฟ้าขนาดใหญ่ได้

งาน	เครือข่าย	ฉันทามติ	กลไกตลาด	แยก off/on-chain
Mengelkamp [10]	Public (LEM)	Public	Local market / bidding	ไม่แยกชัดเจน
Munsing [11]	Public	Public	Decentralized optimization	ไม่แยกชัดเจน
Kang [12]	Consortium	Consortium	Iterative double auction	บางส่วน
Hyperledger Fabric [13]	Permissioned	Pluggable (Raft/BFT)	แพลตฟอร์ม (ไม่เจาะตลาด)	—
Esmat [3]	Public (Ethereum)	Public	Double auction	บางส่วน
งานนี้	Consortium (Solana-compatible)	PoH + Tower BFT (กำกับเข้าร่วมแบบ PoA)	CDA price-time แบ่งโซน	แยกชัดเจน + Ed25519/DLMS

ตารางที่ 2: ตำแหน่งของงานนี้เทียบกับงานวิจัยที่ใช้บล็อกเชนสำหรับตลาดพลังงานแบบ Peer-to-Peer

3. แบบจำลองระบบ ความเชื่อถือ และผู้โจมตี

ส่วนนี้กำหนดแบบจำลองระบบ (system model) ผู้มีส่วนร่วม (actors) สมมติฐานความเชื่อถือ (trust assumptions) และแบบจำลองผู้โจมตี (adversary model) ของระบบจำลอง เพื่อให้ขอบเขตด้านความปลอดภัยของสถาปัตยกรรมแบบแยกชั้นชัดเจนก่อนการอธิบายการออกแบบในหัวข้อที่ 4 ทั้งนี้ขอบเขตของงานเป็นการออกแบบบนระบบจำลอง การวิเคราะห์ด้านล่างจึงระบุทั้งภัยคุกคามที่กลไกปัจจุบันรองรับและความเชื่อถือว่ายังคงเหลืออยู่ (residual trust) อย่างตรงไปตรงมา

3.1 ผู้มีส่วนร่วมในระบบ

ระบบประกอบด้วยผู้มีส่วนร่วมหลักดังนี้

- **Smart Meter (edge device):** อุปกรณ์ปลายทางที่ถือกุญแจคู่ Ed25519 ประจำเครื่อง ลงนามค่าอ่านพลังงานก่อนส่งออก
- **Aggregator Bridge:** ชั้นตรวจสอบและรวบรวมข้อมูลนอกเซน ตรวจสอบลายเซ็น Ed25519 ของทุกค่าอ่านเทียบกับกุญแจสาธารณะของอุปกรณ์ ถอดรหัส payload ตามมาตรฐาน DLMS/COSEM และประเมินเงื่อนไขปริมาณพลังงานคงเหลือและ Grid stability
- **Trading Service:** จับคู่คำสั่งซื้อขายด้วย Continuous Double Auction (CDA)
- **Chain Bridge:** ทำหน้าที่เป็น Reference Monitor และเป็นบริการเดียวที่ถือกุญแจลงนามและเรียก Solana RPC โดยตรงทุกธุรกรรมที่เขียนลงเซนถูกบีบให้ผ่านเส้นทางลงนามเดียว (single signing path)
- **Anchor Programs:** ชั้นบังคับใช้กติกาบนเซนที่รับเฉพาะคู่คำสั่งที่ผ่านการตรวจสอบแล้ว และทำหน้าที่เป็นชั้น settlement และ audit
- **PoA / Governance Authority:** หน่วยงานผู้มีสิทธิ์หลักตามนโยบาย consortium ที่ควบคุมการรับรอง REC สิทธิ์ผู้มีอำนาจ และ allow-list ของ aggregator

3.2 สมมติฐานความเชื่อถือ

สมมติฐานความเชื่อถือของระบบถูกบังคับใช้ในสามขอบเขต ได้แก่ ขอบเขตอุปกรณ์-สู่-บริดจ์ (device-to-bridge) ขอบเขตระหว่างบริการ (service-to-service) และขอบเขตบริการ-สู่-เซน (service-to-chain)

ในขอบเขตอุปกรณ์-สู่-บริดจ์ ตัวตนของอุปกรณ์แต่ละเครื่องผูกกับกุญแจสาธารณะ Ed25519 ที่ลงทะเบียนไว้ Aggregator Bridge ตรวจสอบลายเซ็นของค่าอ่านทุกรายการ (ลายเซ็น 64 ไบต์ กุญแจสาธารณะ 32 ไบต์) ทั้งแบบรายจุดและแบบกลุ่ม (batch) โดยใช้นโยบายแบบ fail-closed กล่าวคือเมื่อค้นกุญแจไม่พบหรือแหล่งเก็บกุญแจไม่ตอบสนอง ระบบจะปฏิเสธ (error) มิใช่ยอมรับค่าอ่านโดยปริยาย ส่วน payload แบบ Binary ตามมาตรฐาน DLMS/COSEM ถูกเข้ารหัสด้วย AES-256-GCM ด้วยกุญแจประจำอุปกรณ์ ในโหมด production หากอุปกรณ์ไม่มีกุญแจเข้ารหัส เฟรมนั้นจะถูกข้าม (fail-closed) และเส้นทาง plaintext เปิดใช้ได้เฉพาะในโหมดพัฒนาเท่านั้นผ่านการตั้งค่าที่ระบุชัด

ในขอบเขตระหว่างบริการที่เขียนสู่เซน (โดยเฉพาะเส้นทางสู่ Chain Bridge) การสื่อสารใช้ ConnectRPC บน mutually authenticated TLS (mTLS) โดยผูกตัวตนของบริการด้วย SPIFFE [34] X.509 identity ที่ตรวจสอบจากใบรับรองฝั่ง client ในขั้นตอน handshake สิทธิ์การเข้าถึงถูกกำหนดจาก SPIFFE URI ของใบรับรองที่ผ่านการตรวจสอบแล้ว แปลงเป็นบทบาทบริการ (service role) เช่น AggregatorBridge, TradingMatcher, SettlementService ทั้งนี้ตัวตนถูกพิจารณาจากชั้น transport (L4) มิใช่ส่วนหัวของแอปพลิเคชัน (L7) ผู้เรียกที่ไม่ผ่านการตรวจสอบจะได้รับบทบาท Unknown อนึ่ง เส้นทางรับข้อมูลมาตรวัดที่ Aggregator Bridge ใช้กลไกพิสูจน์ตัวตนระดับคำขอด้วย API key (ตรวจสอบผ่าน IAM พร้อม static key สำรอง) ประกอบกับลายเซ็น Ed25519 รายค่าอ่าน มิใช่ mTLS/SPIFFE ส่วนการยกเว้นตรวจสอบ (insecure mode) ที่ให้สิทธิ์เต็มมีไว้สำหรับโหมดพัฒนาเท่านั้น

ในขอบเขตบริการ-สู่-เซน Chain Bridge เป็นบริการเดียวที่ถือกุญแจและส่งธุรกรรม โดยกุญแจลงนามถูกจัดเก็บใน HashiCorp Vault Transit (Ed25519) และไม่เคยเข้าสู่หน่วยความจำของกระบวนการ การลงนามถูกจำกัดด้วยชื่อกุญแจที่ได้รับอนุญาตเท่านั้น สำหรับเส้นทางเขียนแบบอะซิงโครนัสผ่าน NATS JetStream [14] ผู้เผยแพร่ (publisher) ต้องลงนามของข้อความ (envelope) ด้วยลายเซ็น ECDSA P-256 เหนือไบนารีแบบ canonical ของของ ซึ่งตรวจสอบเทียบกับกุญแจสาธารณะในใบรับรอง client ของผู้เผยแพร่ และ Chain Bridge ตรวจสอบลำดับ ใบรับรอง → CA → SPIFFE SAN เทียบกับ service identity → ลายเซ็น ก่อนขั้นตอน RBAC โดยหัวข้อที่เคลื่อนย้ายมูลค่า เช่น การ mint โทเคนพลังงาน ถูกบังคับให้ต้องมีของที่ลงนามเสมอ

3.3 แบบจำลองผู้โจมตีและกลไกรองรับ

แบบจำลองผู้โจมตีพิจารณาผู้โจมตีที่สามารถสร้าง ดักจับ หรือส่งซ้ำข้อความบนเครือข่าย และอาจพยายามปลอมตัวตนของอุปกรณ์หรือบริการ แต่ไม่สามารถเข้าถึงกุญแจส่วนตัวที่จัดเก็บใน Vault หรือกุญแจประจำอุปกรณ์ได้ ภัยคุกคามที่พิจารณาและกลไกรองรับสรุปไว้ในตารางที่ 3

ภัยคุกคาม	คำอธิบาย	กลไกรองรับ
T1 คำอ่านปลอม/ส่งซ้ำ	ปลอมหรือเล่นซ้ำคำอ่านมาตรวัด	ตรวจสอบลายเซ็น Ed25519 ต่อคำอ่าน (fail-closed) + order nullifier ในชั้น settlement
T2 ปลอมตัวตนบริการ	แอบอ้างเป็นบริการในเมซ	mTLS + SPIFFE identity → service role; ผู้ไม่ผ่านได้บทบาท Unknown
T3 ปลอม publisher / mint	ส่งคำสั่งเขียนเซนปลอมผ่าน NATS	ซอง envelope ลงนามผูกกับใบรับรอง; หัวข้อ mint บังคับลงนาม
T4 ส่งซ้ำคำสั่งเขียนเซน	ชำระค่าคำสั่งเดิมซ้ำ	บัญชี order nullifier (PDA) — ชำระได้ครั้งเดียว
T5 mint ซ้ำ/เกินสิทธิ์	สร้างโทเคนเกินการผลิตจริง	คีย์ idempotency (meter_id, window) + PDA + การรวมลงนาม REC

ตารางที่ 3: ภัยคุกคามที่พิจารณาและกลไกที่ใช้รองรับ

3.4 ความเชื่อถือที่เหลืออยู่และขอบเขตที่ไม่ครอบคลุม

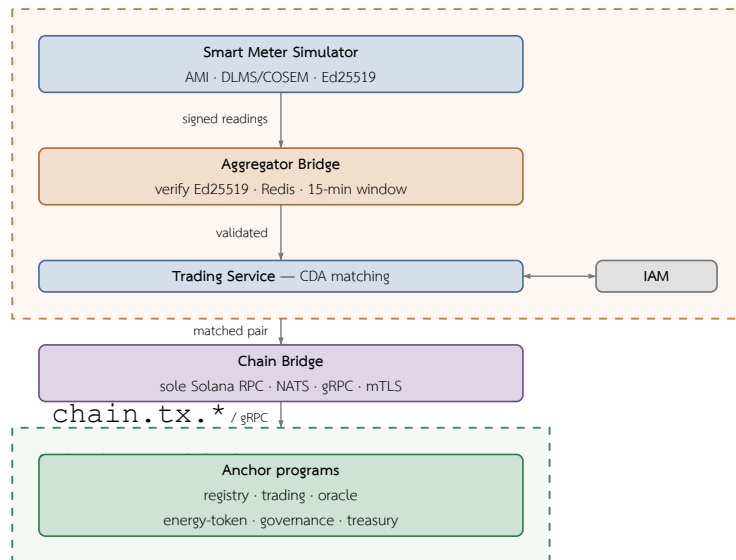
ระบบยังคงมีความเชื่อถือที่เหลืออยู่ (residual trust) ซึ่งต้องระบุอย่างชัดเจน ประการแรก Aggregator Bridge และ governance/oracle authority เป็นผู้รับรองเงื่อนไขปริมาณพลังงานคงเหลือและ Grid stability ในชั้น off-chain โดยเงื่อนไขเหล่านี้ไม่ได้ถูกตรวจสอบซ้ำบนเซน ดังนั้นผู้ใช้ต้องเชื่อถือความถูกต้องของชั้นตรวจสอบนอกเซนนี้ การประนีประนอม (compromise) หรือการสมรู้ร่วมคิด (collusion) ของ Aggregator Bridge หรือ oracle authority ยังไม่ถูกป้องกันด้วยกลไกเชิงรหัสวิทยาในต้นแบบปัจจุบัน แนวทางลด trust เพิ่มเติม เช่น multi-oracle attestation หรือ cryptographic proof เป็นงานในอนาคต ประการที่สอง ความปลอดภัยของชั้นฉันทามติ (PoH ร่วมกับ Tower BFT) ตั้งอยู่บนสมมติฐานเครือข่าย permissioned ที่ควบคุมการเข้าร่วม validator แบบ PoA โดย validator เป็นหน่วยงานได้รับอนุญาตและมีพฤติกรรมตามนโยบาย ซึ่งเป็นสมมติฐานเชิงสถาปัตยกรรมที่ยังไม่ได้ประเมินเชิงปริมาณ ประการที่สาม การยกเว้นตรวจสอบสำหรับโหมดพัฒนา (insecure mode และ plaintext fallback) ต้องถูกปิดในการใช้งานจริง มิฉะนั้นจะทำลายสมมติฐานความเชื่อถือข้างต้นทั้งหมด

4. การออกแบบโครงการ

หลังจากกำหนดแบบจำลองระบบและขอบเขตความเชื่อถือแล้ว ส่วนนี้อธิบายการออกแบบระบบทั้งหมด เริ่มจากภาพรวมสถาปัตยกรรมและบริการฝั่ง Backend (4.1–4.2) ชุดจำลองโครงข่ายและมาตรวัดที่ผลิตข้อมูลนำเข้า (4.3) การออกแบบเครือข่าย consortium (4.4) กลไกราคาและการจับคู่คำสั่ง (4.5–4.6) โปรแกรมบนเซน (4.7) การชำระธุรกรรมแบบ atomic (4.8) แบบจำลองข้อมูลและขอบเขตความรับผิดชอบ (4.9) และวงจรการซื้อขายทั้งระบบ (4.10)

4.1 ภาพรวมสถาปัตยกรรมและขอบเขตความเชื่อถือ

ระบบแบ่งความรับผิดชอบออกเป็นสองโดเมนความเชื่อถือที่เชื่อมต่อกันผ่านจุดเดียว โดเมนนอกเซน (off-chain) ทำหน้าที่ตรวจสอบและจับคู่ ประกอบด้วยชั้นมาตรวัด (Smart Meter Simulator ตามมาตรฐาน AMI และ DLMS/COSEM) ที่ป้อนข้อมูลเข้าสู่ Aggregator Bridge เพื่อยืนยันลายเซ็น Ed25519 และคัดกรองข้อมูล ก่อนส่งต่อให้ Trading Service จับคู่คำสั่งด้วยอัลกอริทึม CDA โดยมี IAM Service และ Notification Service สนับสนุนด้านการระบุตัวตนและการแจ้งเตือน ส่วนโดเมนบนเซน (on-chain) ทำหน้าที่ชำระธุรกรรมและบันทึกหลักฐาน ประกอบด้วยกลุ่มโปรแกรม Anchor หกตัว ได้แก่ registry, trading, oracle, energy-token, governance และ treasury ที่บันทึกสถานะลงบัญชี PDA และ Event log ดังภาพที่ 1



ภาพที่ 1: สถาปัตยกรรมระบบและขอบเขตความเชื่อถือระหว่างโดเมนนอกเชนกับโดเมนบนเชน

การข้ามจากโดเมนนอกเชนไปยังโดเมนบนเชนเกิดขึ้นผ่าน Chain Bridge เพียงทางเดียว ซึ่งเป็นบริการเดียวที่ติดต่อกับ Solana RPC โดยตรง โดยรับคำสั่งเขียนผ่าน NATS JetStream หัวข้อตระกูล `chain.tx.*` และให้บริการอ่านสถานะผ่าน ConnectRPC บนช่องทางที่ป้องกันด้วย mTLS การออกแบบที่แยกสองโดเมนเช่นนี้ทำให้การตรวจสอบเชิงวิศวกรรมไฟฟ้าและการควบคุมสิทธิ์เกิดขึ้นก่อนการบันทึกธุรกรรม ขณะที่บล็อกเชนทำหน้าที่เป็นชั้นชำระธุรกรรมและตรวจสอบย้อนหลังที่บาง (thin settlement and audit layer)

ในด้านกลไกเชิงเศรษฐศาสตร์ ระบบใช้โทเคนสามประเภทที่แยกบทบาทกันชัดเจน คือ โทเคนพลังงาน (energy token) ที่ออกจากการผลิตจริงและรับรองด้วย Renewable Energy Certificate (REC) สำหรับการส่งมอบ เหรียญ THBG ที่ตรึงค่ากับเงินบาทสำหรับการตั้งราคาและชำระเงิน และโทเคน GRX สำหรับการ stake และการกำกับดูแล นโยบายด้านสิทธิ์ถูกบังคับใช้ผ่านโปรแกรม governance บนเชน ครอบคลุมการโอนสิทธิ์ผู้มีส่วนได้ส่วนเสียแบบสองขั้นตอน การควบคุม allow-list ของ Aggregator และการลงคะแนนแบบ DAO เพื่อปรับพารามิเตอร์เชิงเศรษฐศาสตร์ของแต่ละโซน ทั้งนี้ PoA ในที่นี้เป็นชั้นกักกับการเข้าร่วม (admission control) มิใช่การแทนที่กลไกฉันทามติระดับ Layer 1 ซึ่งยังคงอาศัย PoH ร่วมกับ Tower BFT

4.2 บริการฝั่ง Backend

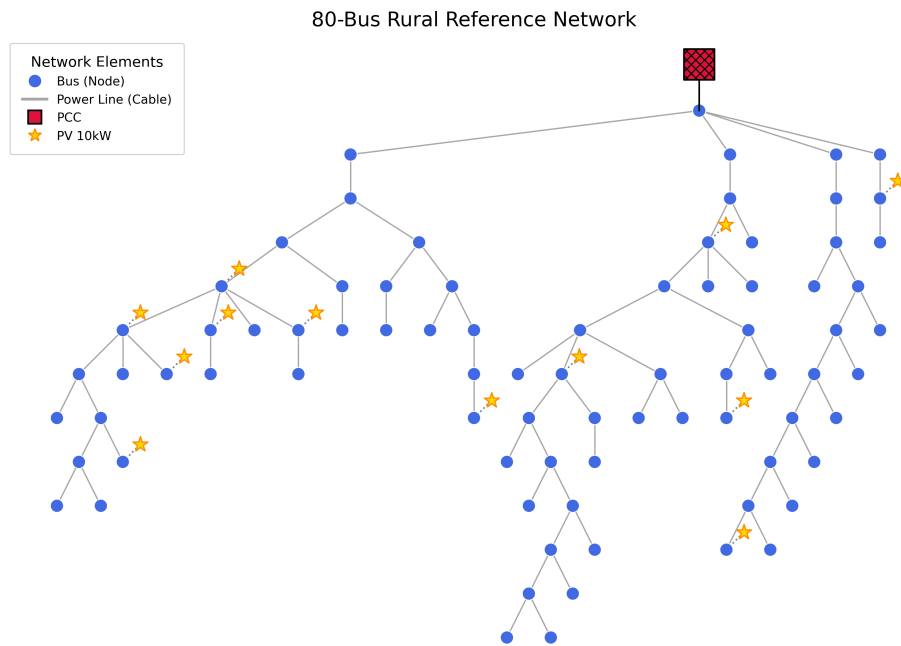
ชั้น Backend ทำหน้าที่เป็นชั้นกลางระหว่าง Smart Meter Simulator, Aggregator Bridge และ Smart Contract โดยรับผิดชอบการรวบรวมข้อมูล การตรวจสอบความถูกต้อง การจัดคิวธุรกรรม และการส่งคำสั่งไปยังบล็อกเชนหลังจากข้อมูลผ่านเงื่อนไขด้าน Grid stability แล้วเท่านั้น การออกแบบใช้แนวคิด Microservice เพื่อแยกภาระงานออกเป็นภาระงานย่อย ทำให้ขยายระบบเฉพาะส่วนที่มีปริมาณค่าสูงและลดผลกระทบเมื่อบริการใดบริการหนึ่งเกิดความผิดพลาด องค์ประกอบหลักของชั้น Backend ประกอบด้วย

- **IAM Service:** จัดการตัวตน สิทธิ์การเข้าถึงข้อมูล On-chain และบทบาทของผู้ใช้งาน เช่น Prosumer, Consumer และ Operator
- **Aggregator Bridge:** รับข้อมูลการผลิตและการใช้ไฟฟ้าจาก Smart Meter ตรวจสอบรูปแบบข้อมูล เวลาอ้างอิง รหัสอุปกรณ์ และค่าพลังงานก่อนส่งต่อไปยังขั้นตอนวิเคราะห์
- **Trading Service:** ตรวจสอบและจับคู่คำสั่งซื้อขายพลังงานร่วมกับเงื่อนไขด้าน Grid stability เช่น ปริมาณพลังงานคงเหลือ ข้อจำกัดของโครงข่าย และสถานะการเชื่อมต่อของ Smart Meter
- **Chain Bridge:** ตัวกลางเดียวที่ส่งคำสั่งไปยัง Solana Program และติดตามผลลัพธ์จาก transaction signature หรือ Event log โดยบริการอื่นไม่เรียก Solana RPC โดยตรง
- **Notification Service:** ส่งการแจ้งเตือน เช่น Email และ Alert เมื่อคำสั่งซื้อขายหรือสถานะ settlement เปลี่ยนแปลง

การแยกบริการในลักษณะนี้ช่วยให้ระบบรองรับข้อมูล Realtime Smart Meter จำนวนมากได้ดีขึ้น เนื่องจากบริการรับข้อมูลขยายจำนวน instance ได้โดยไม่กระทบต่อการบริการตรวจสอบธุรกรรมหรือบริการเชื่อมต่อบล็อกเชน นอกจากนี้ชั้น Backend ยังเป็นจุดควบคุมความปลอดภัยก่อนบันทึกธุรกรรมลง Smart Contract ทำให้ระบบไม่พึ่งพาล็อกเชนเพียงอย่างเดียว แต่ผสานการตรวจสอบทางวิศวกรรมไฟฟ้าและการควบคุมสิทธิ์ของผู้ใช้งานเข้าด้วยกัน ส่วน Settlement Engine ทำหน้าที่จัดการธุรกรรมที่ลงนามแล้ว ติดตามผลจาก transaction signature และอ่าน Event log เพื่อนำสถานะกลับไปแสดงบน Frontend

4.3 ชุดจำลองโครงข่ายและมาตรวัด

ชุดจำลองใช้โครงสร้าง Network Topology จากไฟล์ GridLAB-D GLM [19] แล้วแปลงเป็นแบบจำลองโครงข่ายที่ประกอบด้วย bus, line, load และ photovoltaic unit ดังภาพที่ 2 การจำลองทำงานแบบ discrete time-step โดยแต่ละรอบประกอบด้วยการสร้างค่าอ่านที่ระดับมาตรวัด แล้วตามด้วยการแก้สถานะโครงข่ายที่ระดับ grid



ภาพที่ 2: โครงสร้างโครงข่าย bus ที่ใช้ในชุดจำลอง

ในแต่ละ bus ระบบสร้างประชากรมาตรวัดตามสัดส่วนชนิดของผู้ใช้ โดยมาตรวัดแต่ละตัวประกอบขึ้นจากแบบจำลองอุปกรณ์ย่อย ได้แก่ โหลด แผงเซลล์แสงอาทิตย์เมื่อมีการติดตั้ง และระบบกักเก็บพลังงานแบตเตอรี่แบบเลือกได้ พฤติกรรมการใช้ไฟฟ้าจำลองด้วยแบบจำลองโหลดแบบ ZIP ที่ตอบสนองต่อแรงดัน ตามสมการ

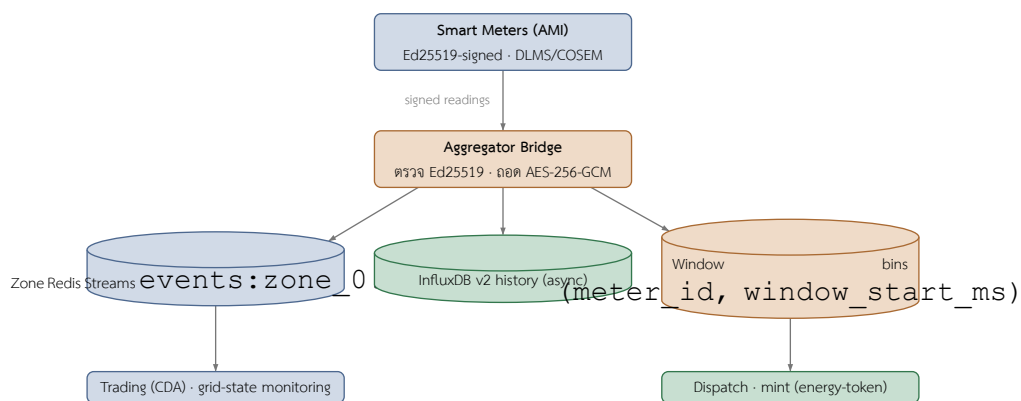
$$P(V) = P_{\text{base}}(Z \cdot V^2 + I \cdot V + P), \quad Z + I + P = 1$$

โดยองค์ประกอบ Z แปรผันตาม V^2 , องค์ประกอบ I แปรผันตาม V และองค์ประกอบ P คงที่ ส่วนกำลังผลิตจากแผงเซลล์แสงอาทิตย์คำนวณด้วย pvlib [17] โดยใช้แบบจำลองท้องฟ้าใสแบบ Ineichen ร่วมกับแบบจำลอง PVWatts แล้วลดทอนด้วยตัวประกอบสภาพอากาศ เพื่อให้การจำลองทำซ้ำได้ มาตรวัดแต่ละตัวสุ่มสัญญาณรบกวนจากสตรึมเฉพาะของตน โดยเฉพาะค่าเริ่มต้นของตัวสุ่มจากการ XOR ระหว่าง seed รวมของระบบกับไจเจสต์ SHA-256 ของรหัสมาตรวัด ทำให้การเพิ่มหรือลบมาตรวัดหนึ่งตัวไม่ทำให้ค่าอ่านของมาตรวัดตัวอื่นเคลื่อน ก่อนส่งออก มาตรวัดแต่ละตัวลงนามด้วยกุญแจ Ed25519 เฉพาะตัวบนสายอักขระตามแบบแผน device_id:kwh:timestamp_ms ทำให้ตรวจสอบที่มาของค่าอ่านได้รายจุดโดยไม่ต้องเก็บกุญแจลับไว้ที่ส่วนกลาง

เมื่อรวบรวมค่าอ่านของมาตรวัดทุกตัวแล้ว ระบบคำนวณการอัดกำลังสุทธิของแต่ละ bus จากผลต่างระหว่างการผลิตและการใช้ไฟฟ้า แล้วแก้ปัญหา power flow ด้วย pandapower [18] แบบ backward/forward sweep เพื่ออัปเดตแรงดันต่อหน่วยของ bus การไหลของกำลังในสาย กำลังสูญเสีย และระดับการใช้งานของสาย และเมื่อการคำนวณไม่ลู่เข้า ระบบจะถอยไปใช้ตัวแก้แบบประมาณ DistFlow เป็นค่าสำรองเพื่อให้การจำลองดำเนินต่อไปได้

นอกจากการแก้ power flow พื้นฐาน ชั้น grid simulation ยังจำลองกลไกควบคุมแรงดันและเหตุการณ์ผิดปกติของโครงข่ายจำหน่าย ได้แก่ การตอบสนองของอินเวอร์เตอร์แบบ volt-watt และ volt-VAR การปรับแทปหม้อแปลงแบบ on-load tap changer (OLTC) การฉีดความผิดปกติ (fault injection) ที่สาย bus หรือหม้อแปลง และการสับสวิตช์เชื่อมโยง (tie-switch) เพื่อถ่ายโอนโหลด ทำให้ชุดข้อมูลที่ส่งออกไปยัง Aggregator Bridge สะท้อนพฤติกรรมเชิงฟิสิกส์ของโครงข่ายภายใต้สภาวะการทำงานและสภาวะผิดปกติที่หลากหลาย มิใช่เพียงค่าพลังงานที่สุ่มขึ้นอย่างอิสระ ทั้งนี้ความสามารถด้านฟิสิกส์ของโครงข่ายข้างต้นเป็นส่วนหนึ่งของชุดจำลอง แต่ยังไม่ได้ใช้เป็นตัวแปรในการประเมินที่รายงานในบทความนี้

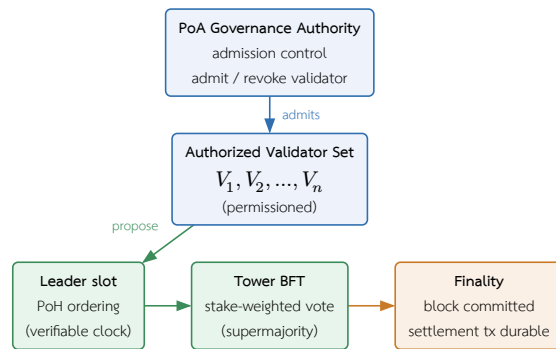
เพื่อให้เป็นไปตามมาตรฐานอุตสาหกรรมและรับรองที่มาของข้อมูล Aggregator Bridge รับข้อมูลมาตรวัดความถี่สูงแบบเวลาจริงแล้วกระจายผ่าน Redis Streams ที่แบ่งตามโซน (zone-partitioned) สำหรับการเฝ้าระวังสถานะโครงข่ายแบบพลวัต พร้อมรวมค่าอ่านเป็นหน้าต่างเวลา 15 นาทีเพื่อใช้ในการประเมินกำลังการผลิตและการสั่งการ (dispatch) ในชั้น Backend และเขียนประวัติแบบเวลาจริงลง InfluxDB แบบอะซิงโครนัส รูปแบบข้อมูลมาตรวัดเป็นไปตามมาตรฐาน DLMS/COSEM (IEC 62056) โดยถอดรหัสส่วน payload แบบ Binary ด้วย AES-256-GCM และเผยแพร่ต่อในรูปแบบ JSON นอกจากนี้ ระบบรับประกันความไม่ปฏิเสธเชิงรหัสวิทยา (Cryptographic Non-repudiation) ด้วยการตรวจสอบลายเซ็น Ed25519 ทั้งแบบค่าอ่านรายจุดและแบบกลุ่ม (Batch) ก่อนรับข้อมูลเข้าสู่ระบบ เส้นทางการกระจายข้อมูลของ Aggregator Bridge สรุปไว้ในภาพที่ 3



ภาพที่ 3: เส้นทางการกระจายข้อมูล telemetry ของ Aggregator Bridge: ค่าอ่านที่ผ่านการตรวจสอบแล้วกระจายเข้าสู่ Redis Streams รายโซน (เวลาจริง), InfluxDB สำหรับประวัติ (อะซิงโครนัส) และถึงรวมหน้าต่าง 15 นาทีที่ผูกกับคู่ (meter_id, window_start_ms)

4.4 เครือข่าย Consortium และการกำกับดูแล

เครือข่ายบล็อกเชนในระบบนี้ออกแบบเป็น Consortium Network ภายใต้สมมติฐานการกำกับดูแลแบบ Proof of Authority (PoA) โดยผู้ตรวจสอบบล็อกเป็นหน่วยงานที่มีหน้าที่ตรวจสอบหรือได้รับความยินยอมจาก DSO เช่น ผู้ให้บริการรวบรวมโหลด (Load Aggregator) หน่วยงานกำกับดูแล หรือองค์กรที่ได้รับอนุญาต เหตุผลหลักในการเลือก PoA คือ Network Governance ที่ควบคุมได้ง่ายกว่าเครือข่ายสาธารณะ เช่น Solana Mainnet หรือ Ethereum และความเหมาะสมต่อข้อกำหนดด้าน Regulatory compliance และ cost predictability สำหรับระบบพลังงานที่ต้องประมาณต้นทุนธุรกรรมได้ล่วงหน้า [27] [13] สิ่งสำคัญที่ต้องเน้นย้ำคือ PoA ในที่นี้เป็นชั้น governance และการควบคุมสิทธิ์การเข้าร่วม (admission control) ไม่ใช่การแทนที่กลไกฉันทามติระดับ Layer 1 ของเครือข่าย Solana-compatible ซึ่งยังคงอาศัย PoH ร่วมกับ Tower BFT ในการเรียงลำดับและประกาศ finality การแบ่งบทบาทดังกล่าวแสดงในภาพที่ 4



ภาพที่ 4: การแยกบทบาทในเครือข่าย consortium: ชั้น PoA กำกับกับการรับเข้าชุด validator ที่ได้รับอนุญาต ขณะที่ฉันทามติระดับ Layer 1 ยังคงเป็น PoH สำหรับเรียงลำดับเวลาและ Tower BFT สำหรับการลงมติและ finality

นโยบายด้านสิทธิ์และการกำกับดูแลถูกบังคับใช้ผ่านโปรแกรม governance บนเชน ซึ่งครอบคลุมสามกลไกหลัก กลไกแรกคือการกำกับหน่วยงานผู้มีสิทธิ์หลัก (PoA authority) ผ่านการโอนสิทธิ์แบบสองขั้นตอน (two-step handover) โดย `propose_authority_change` ให้ผู้มีสิทธิ์ปัจจุบันกำหนด pending authority พร้อมเวลาหมดอายุ และ `approve_authority_change` ต้องถูกเรียกโดย pending authority เองเพื่อรับโอน ทำให้ไม่สามารถยกเลิกสิทธิ์ให้บัญชีที่ไม่ยินยอมหรือผิดพลาดได้ และอนุญาตให้มูลค่าขอเปลี่ยนแปลงได้เพียงรายการเดียวต่อครั้ง กลไกที่สองคือการควบคุม allow-list ของ Aggregator ที่ได้รับอนุญาตให้ลงนามค่าอ่านผ่าน `admit_aggregator` และ `revoke_aggregator` กลไกที่สามคือการลงคะแนนแบบ DAO ผ่าน `create_proposal`, `cast_vote` และ `execute_proposal` สำหรับปรับพารามิเตอร์เชิงเศรษฐศาสตร์ของแต่ละโซน (zone_config) ได้แก่ incentive multiplier, wheeling charge, loss factor และ maintenance mode

การลงคะแนนใช้น้ำหนักตามกำลังการผลิตสะสมของมาตรวัด (stake-by-generation) โดยน้ำหนักของผู้ลงคะแนนคำนวณเป็น $w = \max\left(100, \frac{\text{total generation}}{1000}\right)$ กล่าวคือทุก 1,000 kWh ของการผลิตสะสมเทียบเท่าหนึ่งหน่วยน้ำหนัก โดยมีค่าขั้นต่ำ 100 เพื่อให้ผู้เข้าร่วมรายเล็กยังมีเสียง การกันลงคะแนนซ้ำใช้บัญชี vote record แบบ PDA ต่อคู่ (proposal, voter) หนึ่งบัญชีต่อหนึ่งเสียง เมื่อสิ้นสุดช่วงเวลาลงคะแนน `execute_proposal` จะสรุปผลแบบอัตโนมัติ ข้อเสนอจะผ่าน (Passed) ก็ต่อเมื่อผลรวมคะแนนถึงเกณฑ์องค์ประชุมขั้นต่ำ (min_quorum_votes ที่กำหนดใน GovernanceConfig) และคะแนนเห็นด้วยมากกว่าคะแนนไม่เห็นด้วย มิฉะนั้นถือว่าตกไป (Rejected) จึงป้องกันการเปลี่ยนพารามิเตอร์ด้วยผู้ลงคะแนนจำนวนน้อยเกินไป

ความแตกต่างจาก Solana public mainnet คือเครือข่ายนี้ไม่เปิดให้ validator ภายนอกเข้าร่วมแบบ permissionless และสามารถกำหนดนโยบายด้านสิทธิ์ การอัปเดตโปรแกรม และการเก็บข้อมูลตามข้อกำหนดของโครงการได้ อย่างไรก็ตาม execution layer ยังคงอ้างอิงสถาปัตยกรรม Solana Virtual Machine และ Sealevel parallel runtime เพื่อให้ธุรกรรมที่ไม่ใช่ account เดียวกันสามารถประมวลผลแบบขนานได้ ค่า slot time ใกล้ 400 มิลลิวินาทีและ compute budget ที่กล่าวถึงในบทความนี้เป็นเป้าหมายเชิงออกแบบที่อ้างอิงเอกสาร Solana [31] ไม่ใช่ผลวัดของเครือข่าย permissioned ทั้งนี้ในระดับเครือข่ายดังกล่าวต้องกำหนดอย่างน้อยห้ารายการก่อนนำไปใช้งานจริง ได้แก่ จำนวน Validator และหน่วยงานเจ้าของ โหนด วิธีผูก identity กับกุญแจ Validator นโยบายเพิ่มหรือลบ Validator กระบวนการอัปเดตโปรแกรมหรือ genesis/configuration และ fault model ที่ยอมรับได้ บทความนี้จึงถือว่า PoA Solana-compatible network เป็นสมมติฐานเชิงสถาปัตยกรรมของระบบจำลอง ไม่ใช่ข้อสรุปว่ามีการปรับแต่ง consensus ของ Solana public network แล้ว

4.5 กลไกราคาและการคำนวณการชำระธุรกรรม

แบบจำลองราคาของระบบแบ่งเป็นสามส่วน คือ การกำหนดราคาเคลียร์ในกลไก CDA การคำนวณค่าธรรมเนียมและยอดสุทธิในการ settlement และกลไกราคาของโทเคนในชั้น treasury สัญลักษณ์ที่ใช้ในสมการสรุปไว้ในตารางที่ 4

สัญลักษณ์	ความหมาย	สัญลักษณ์	ความหมาย
-----------	----------	-----------	----------

p_s	ราคาเสนอขายต่อหน่วย (sell ask)	V	มูลค่ารวมของธุรกรรม
p_b	ราคาเสนอซื้อต่อหน่วย (buy bid)	f	ค่าธรรมเนียมตลาด
p^*	ราคาเคลียร์ / landed cost	φ	อัตราค่าธรรมเนียมตลาด (bps)
λ	loss factor ($\lambda \geq 1$)	W	ค่าผ่านสายรวม ($q \cdot w$)
c_{loss}	ต้นทุนการสูญเสียต่อหน่วย	L	ต้นทุนการสูญเสียรวม ($q \cdot c_{\text{loss}}$)
w	ค่าผ่านสาย (wheeling charge) ต่อหน่วย	r	อัตราแลกเปลี่ยน (หน่วยย่อย THBG ต่อ 1 GRX)
m	ตัวคูณจูงใจ (incentive multiplier)	ψ	ค่าธรรมเนียม swap (bps)
δ	ส่วนลดภายในโซน (intra-zone discount)	A	ตัวสะสมรางวัลต่อหน่วย stake
q	ปริมาณพลังงานที่จับคู่ (kWh)	s, s_{total}	จำนวนที่ stake และ stake รวม

ตารางที่ 4: สัญลักษณ์ที่ใช้ในการราคาและการชำระธุรกรรม

การกำหนดราคาเคลียร์ (CDA clearing) ใช้ราคาฝั่งผู้ขาย (maker price) ปรับด้วยต้นทุนโครงข่าย โดยกำหนดต้นทุนการสูญเสีย (loss cost) ต่อหน่วยจากค่า loss factor $\lambda \geq 1$ ดังสมการ

$$c_{\text{loss}} = p_s (\lambda - 1)$$

ราคาเคลียร์หรือ landed cost คำนวณจากราคาเสนอขาย p_s บวกค่าผ่านสาย (wheeling charge) w และต้นทุนการสูญเสีย แล้วปรับด้วยตัวคูณจูงใจ m และส่วนลดภายในโซน δ

$$p^* = (p_s + w + c_{\text{loss}}) \cdot m \cdot \delta$$

โดย $\delta = 0.95$ เมื่อผู้ซื้อและผู้ขายอยู่ในโซนเดียวกัน และ $\delta = 1$ เมื่อข้ามโซน คำสั่งซื้อจะจับคู่ได้เมื่อ $p^* \leq p_b$ (เมื่อ p_b คือราคาเสนอซื้อ) และระบบจับคู่ลำดับผู้ขายที่มี landed cost ต่ำสุดให้จับคู่ก่อนตามหลัก price-time priority ปริมาณที่จับคู่เท่ากับส่วนที่เหลือน้อยที่สุดของทั้งสองฝั่ง $q = \min(q_b, q_s)$ จากนั้นการคำนวณมูลค่ารวม ค่าธรรมเนียมตลาด และยอดสุทธิที่ผู้ขายได้รับเป็นดังนี้

$$V = q \cdot p^*, \quad f = \frac{V \cdot \varphi}{10000}, \quad \text{net} = V - f - W - L$$

โดย φ คือค่าธรรมเนียมตลาดในหน่วย basis point (ค่าตั้งต้นบนเซน 25 bps หรือ 0.25%), $W = q \cdot w$ คือค่าผ่านสายรวม และ $L = q \cdot c_{\text{loss}}$ คือต้นทุนการสูญเสียรวม โดยค่าตั้งต้นของ wheeling charge เท่ากับ 0 ภายในโซนและ 0.02 ข้ามโซน ส่วน loss factor เท่ากับ 1.01 ภายในโซนและ 1.03 ข้ามโซน ตัวอย่างการจับคู่ภายในโซนเดียวกันที่ $p_s = 4.00$ บาทต่อ kWh, $q = 10$ kWh และใช้ค่าตั้งต้นภายในโซน ได้ $c_{\text{loss}} = 0.04$, ราคาเคลียร์ $p^* = 3.838$ บาทต่อ kWh, มูลค่ารวม $V = 38.38$ บาท, ค่าธรรมเนียม $f \approx 0.096$ บาท และยอดสุทธิของผู้ขาย $\text{net} \approx 37.88$ บาท เมื่อเทียบกับกรับซื้อไฟส่วนเกินแบบอัตราคงที่ที่สมมติไว้ราว 2.20 บาทต่อ kWh ยอดสุทธิต่อหน่วยของผู้ขายในตลาด P2P (ประมาณ 3.76–3.99 บาทต่อ kWh) คิดเป็นส่วนเพิ่มประมาณ 72–81% ขณะที่ผู้ซื้อจ่ายราคา landed (ประมาณ 3.84–4.22 บาทต่อ kWh) ต่ำกว่าค่าไฟขายปลีกตามปกติ จึงเกิดส่วนเกินจากการซื้อขาย (gains from trade) ทั้งสองฝั่ง (ทั้งนี้อัตรา 2.20 บาทเป็นค่าอ้างอิงเชิงสมมติ มิใช่ค่าที่วัดจากตลาดจริง)

กลไกราคาของโทเคนในชั้น treasury ครอบคลุมการแลกเปลี่ยนระหว่าง GRX และ THBG stablecoin ที่ตรึงค่ากับเงินบาท โดยใช้อัตรา r (หน่วยย่อย THBG ต่อ 1 GRX) และค่าธรรมเนียม swap ψ (bps)

$$\text{thbg} = \frac{g \cdot r}{10^9} \cdot (1 - \psi/10000), \quad g = \frac{\text{thbg} \cdot 10^9}{r}$$

โดยการ redeem ไม่มีค่าธรรมเนียม และการรักษาค่าตรึงใช้เงื่อนไข supply ต่อ reserve แบบ 1:1 คือ $\text{supply}_{\text{thbg}} \leq \text{reserve}_{\text{attested}}$ ส่วนรางวัลการ stake ใช้ตัวสะสม (accumulator) แบบ MasterChef โดยรางวัลค้างรับของผู้ stake คำนวณจากจำนวนที่ stake s เป็น $\text{reward} = s \cdot A / 10^{12} - \text{debt}$ เมื่อมีการเติมรางวัล R ตัวสะสม A จะถูกปรับเป็น $A \leftarrow A + (R \cdot 10^{12}) / s_{\text{total}}$ แบบ pro-rata ตามสัดส่วนการ stake และการ slash จะหักจำนวนที่ร้องขอแต่ไม่เกินเงินต้นที่ stake ไว้ (capped ที่ principal) แล้วกระจายคืนสู่ผู้ stake ที่เหลือผ่านตัวสะสมเดียวกัน

การวิเคราะห์ความไวของยอดสุทธิ

เพื่อแสดงนัยเชิงเศรษฐศาสตร์ของแบบจำลองราคาข้างต้น ส่วนนี้วิเคราะห์ความไว (sensitivity) ของยอดสุทธิที่ผู้ขายได้รับต่อพารามิเตอร์ของโซนและค่าธรรมเนียม โดยเป็นการคำนวณเชิงแบบจำลองล้วน (model-derived) จากสมการ settlement มีใช้การวัดรายได้จากการรันระบบจริง กำหนดราคาเสนอขายฐาน $p_s = 4.00$ บาทต่อ kWh ปริมาณจับคู่ $q = 10$ kWh และตัวคูณจูงใจ $m = 1$ (ตามเส้นทาง CDA จริง) คงที่ แล้วแปรค่าสถานะภายในโซน/ข้ามโซน อัตราค่าธรรมเนียม φ และ loss factor λ ตามตารางที่ 5

สถานการณ์	δ	w	λ	φ (bps)	p^*	net (฿)	net/kWh
S1 ภายในโซน	0.95	0.00	1.01	25	3.838	37.88	3.79
S2 ข้ามโซน	1.00	0.02	1.03	25	4.140	41.30	4.13
S3 ภายในโซน, ค่าธรรมเนียมสูง	0.95	0.00	1.01	100	3.838	37.60	3.76
S4 ข้ามโซน, loss สูง	1.00	0.02	1.05	25	4.220	41.10	4.11

ตารางที่ 5: ความไวของยอดสุทธิที่ผู้ขายได้รับ คำนวณจากสมการ settlement ที่ $p_s = 4.00$ บาท/kWh, $q = 10$ kWh, $m = 1$ (ค่าผ่านสายและ loss เป็นต้นทุนส่งผ่านไปยังผู้ซื้อผ่านราคา landed p^*)

จากตารางที่ 5 เห็นรูปแบบสำคัญสองประการ ประการแรก ยอดสุทธิของผู้ขายแทบไม่เปลี่ยนเมื่อเพิ่มค่าผ่านสาย w หรือ loss factor λ (เทียบ S2 กับ S4 ที่ต่างกันในระดับเศษสตางค์) เพราะต้นทุนทั้งสองถูกบวกเข้าไปในราคา landed p^* ที่ผู้ซื้อจ่าย แล้วถูกแยกออกไปยังบัญชีผู้เก็บค่าผ่านสายและการสูญเสีย จึงเป็นต้นทุนแบบส่งผ่าน (pass-through) ที่ไม่ลดทอนยอดผู้ขายโดยตรง ผู้ขายจึงได้รับยอดใกล้เคียง $q \cdot p_s$ เสมอ ประการที่สอง ตัวแปรที่กระทบยอดผู้ขายอย่างมีนัยคือส่วนลดภายในโซน δ และอัตราค่าธรรมเนียม φ โดยการจับคู่ภายในโซน ($\delta = 0.95$) ลดยอดผู้ขายลงประมาณ 5% เพื่อจูงใจการบริโภคในพื้นที่ ขณะที่การขึ้นค่าธรรมเนียมจาก 25 เป็น 100 bps (S1 $\rightarrow$ S3) ลดยอดสุทธิเพียงเล็กน้อย

ในเส้นทาง CDA ที่ใช้งานจริง ตัวคูณจูงใจถูกตั้งเป็น $m = 1$ กล่าวคือ incentive multiplier มีผลเฉพาะเส้นทาง settlement แบบ feed-in หรือ grid-export มีใช้การจับคู่ CDA นอกจากนี้ค่าตั้งต้นของค่าธรรมเนียมตลาดบนเซน (25 bps) แตกต่างจากค่าตั้งต้นในไฟล์ตั้งค่านอกเซน (50 bps) และพารามิเตอร์โซนในโปรแกรม governance จัดเก็บแบบสเกล $\times 1000$ ขณะที่ผู้บริโภคค่าจริงในชั้น trading ดีความแบบ bps ($/10000$) ซึ่งเป็นค่าที่ระบบใช้งานจริง

4.6 การจับคู่คำสั่งแบบ Continuous Double Auction

เครื่องจับคู่แบ่งสมุดคำสั่งขายตามโซน (zone-segmented) โดยแต่ละโซนเก็บคำสั่งขายในโครงสร้างเรียงลำดับ (BTreeMap) ด้วยกุญแจ (price, created_at, id) จึงได้ price-time priority โดยปริยายโดยไม่ต้องเรียงลำดับซ้ำ สำหรับคำสั่งซื้อแต่ละรายการ เครื่องจับคู่รวบรวมผู้ขายที่เป็นผู้สมัครจากเฉพาะโซนที่โครงข่ายส่งพลังงานถึงโซนของผู้ซื้อได้ ผ่านการตรวจ topology pre-filtering สองชั้น (ที่ปริมาณขั้นต่ำเพื่อตัดโซนที่ส่งถึงกันไม่ได้ออกทันที และตรวจซ้ำที่ปริมาณจับคู่จริงเพื่อบังคับเพดานความจุสายส่ง) แล้วเรียง candidate ตาม landed cost จากต่ำไปสูงเพื่อให้ผู้ซื้อได้ราคารวมส่งถึงที่ถูกลงที่สุดก่อน ระบบยังป้องกันการจับคู่กับตนเอง (self-trade) และรองรับคำสั่งแบบ Fill-or-Kill โดยเครื่องจับคู่ปล่อยผลการจับคู่แต่ละรายการออกโดยตรง ไม่มีขั้นรวมรายการ (no consolidation step) เนื่องจากคำสั่งขายแต่ละรายการอยู่ในสมุดของโซนเดียวกัน ผู้ซื้อ-ผู้ขายหนึ่งคู่จึงให้ผลลัพธ์ไม่เกินหนึ่งรายการต่อรอบจับคู่

4.7 โปรแกรม Smart Contract บนเซน

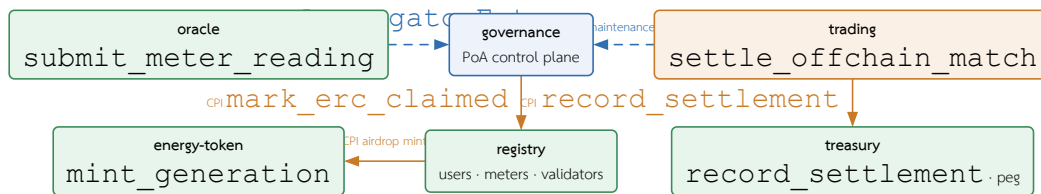
Smart Contract แบ่งออกเป็นหลายโปรแกรมตามความรับผิดชอบ ได้แก่ registry, trading, oracle, energy-token, governance และ treasury รวมถึงโปรแกรมสำหรับการทดสอบประสิทธิภาพ (blockbench และ tpc-benchmark) ซึ่งพัฒนาด้วย Anchor Framework [15] เพื่อกำหนด account validation, instruction handler และ Event log อย่างเป็นระบบ รหัสโปรแกรมบนเซน (program ID) ของโปรแกรมหลักทั้งหกซึ่งประกาศด้วย declare_id! สรุปไว้ในตารางที่ 6

โปรแกรม	Program ID (declare_id!)
registry	FcSd5x4X1nzJMKLZC4tMZXnQ1ipLrGsEfeoH8N4mvJX7
trading	CnWDEUhTvSiXeLSyViWgAnnu9YouBAYVGcrrFm1s9WcX
oracle	64Vgos61STZ8pW9NnHi2iGtXMTQr7NqBoMorK6Zg8RJU
energy-token	6FZKcVKCLFSNLMxypFJGU4K14xUBnxNW9VAuKGhmjGX
governance	FokVuBSPXP11aeL7VZWd8n8aVAhWqVpyPZETToSxdvTS
treasury	FfxSQYKUm9NGdCC9TDPmZSYjWYE1h4ruu3JatzHN5Tn

ตารางที่ 6: รหัสโปรแกรมบนเซนของโปรแกรมหลักทั้งหกบนเครือข่าย Solana-compatible consortium

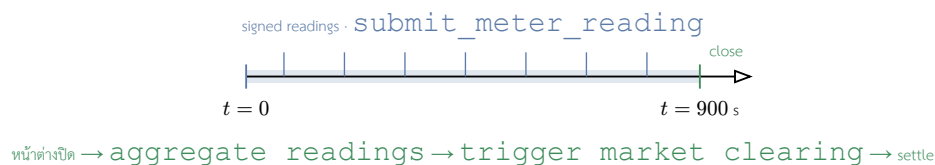
โครงสร้างบัญชีใช้ Program Derived Address (PDA) [32] เพื่อแยก state ของระบบออกเป็นบัญชีย่อยตาม seed เฉพาะ เช่น registry และ user account, meter account, market และ market_shard, order และ trade record, escrow, order nullifier สำหรับป้องกันการส่งซ้ำ, oracle_data และ mint authority การใช้ PDA ทำให้โปรแกรมตรวจสอบ ownership และ deterministic address ของบัญชีได้โดยไม่ต้องพึ่ง private key ของบัญชี state แต่ละรายการ งานที่ใช้ทรัพยากรสูง เช่น การคำนวณราคาแบบซับซ้อน การจับคู่ order book ขนาดใหญ่ หรือการวิเคราะห์ Grid stability จึงถูกย้ายไปทำใน Backend และ Aggregator Bridge ก่อนส่งผลลัพธ์ที่ยืนยันแล้วเข้าสู่ Smart Contract เพื่อให้อยู่ภายใต้ข้อจำกัดด้าน compute ของธุรกรรม [31]

ความสัมพันธ์ระหว่างโปรแกรมทั้งหกแบ่งเป็นสองชนิด คือการเรียกข้ามโปรแกรม (Cross-Program Invocation: CPI) ที่เขียนสถานะ และการอ่านสถานะแบบควบคุม (control-plane read) ที่โปรแกรมหนึ่งอ่านบัญชีของอีกโปรแกรมเพื่อกำหนดสิทธิ์การทำงานโดยไม่เรียก CPI ดังสรุปในภาพที่ 5



ภาพที่ 5: ความสัมพันธ์ระหว่างโปรแกรม Anchor ทั้งหมด เส้นทึบคือ CPI ที่เขียนสถานะ เส้นประคือการอ่านสถานะแบบ control-plane ที่ไม่เรียก CPI

หน้าต่างเวลาการเคลียร์ตลาดกำหนดไว้ที่ 15 นาที (900 วินาที) โดยค่าอ่านที่ลงนามไหลเข้าอย่างต่อเนื่องภายในหน้าต่าง เมื่อหน้าต่างปิดโปรแกรม oracle จึงรวมค่าอ่าน (aggregate_readings) แล้วส่งเคลียร์ตลาด (trigger_market_clearing) ก่อนค่าส่งที่จับคู่แล้วจะถูกชำระบนเซน ดังภาพที่ 6



ภาพที่ 6: ลำดับเวลาการเคลียร์ตลาด: ค่าอ่านที่ลงนามไหลเข้าต่อเนื่องตลอดหน้าต่างรวมข้อมูล 15 นาที (900 วินาที) เมื่อหน้าต่างปิดจึงรวมค่าอ่าน ส่งเคลียร์ตลาด และชำระค่าส่งบนเซน

4.7.1 Trading: วงจรคำสั่งซื้อขายและ escrow บนเชน

โปรแกรม trading จัดการวงจรชีวิตของคำสั่งซื้อขายและบัญชี escrow บนเชน ผู้ใช้สร้างคำสั่งผ่าน `create_sell_order`, `create_buy_order` และ `submit_limit_order` ซึ่งสร้างบัญชี Order แบบ PDA รายคำสั่งที่ผูกกับเจ้าของ (seed [order, authority, order_id]) พร้อมกำหนดอายุคำสั่ง (`expires_at`) ไว้ที่ 24 ชั่วโมง (86,400 วินาที) นับจากเวลาสร้าง ส่วน `submit_market_order` ไม่เปิดบัญชี Order บนเชนและไม่มีเวลาหมดอายุ แต่ปล่อยเหตุการณ์ `MarketOrderSubmitted` ให้จับคู่ในชั้น off-chain แทน ทั้งนี้ใบรับรอง ERC เป็นตัวเลือกเสริมสำหรับคำสั่งขาย (optional) เมื่อแนบมาต้องมีสถานะ Valid ยังไม่หมดอายุ และผ่าน `validate_erc_for_trading` แล้ว ก่อนเข้าสู่ตลาด ผู้ใช้ฝากโทเคนเข้าบัญชี escrow ผ่าน `deposit_escrow` และถอนคืนส่วนที่ไม่ถูกใช้ผ่าน `withdraw_escrow` โดยบัญชี escrow เป็น SPL token account ภายใต้อาณัติ PDA ที่ลงนามโดย `market_authority`

การจับคู่คำสั่งมีสองเส้นทางที่เสริมกัน เส้นทางแรกคือเครื่องจับคู่ในชั้น off-chain (trading-engine) ที่รัน CDA แบบ price-time priority เหนือสมุดคำสั่งทั้งโซนด้วยปริมาณงานสูง เส้นทางที่สองคือคำสั่ง `match_orders` บนเชนที่ตรวจและบันทึกการจับคู่รายคู่ โดยแต่ละจะเฉพาะบัญชี Order สองรายการกับ `zone_market` แล้วเขียน trade record โดยไม่เคลื่อนย้ายโทเคน จึงมีต้นทุน compute ต่ำ ส่วนคำสั่ง `cancel_order` ลบคำสั่งที่ยังค้างในสมุด บัญชีระดับตลาดถูกแยกออกเป็นบัญชี `zone_market` รายโซนที่เก็บความลึกของสมุดคำสั่ง ความจุสายส่ง และการไหลที่ผูกพันแล้ว (`committed_flow`) แยกออกจากบัญชี Market กลาง เพื่อให้การส่งคำสั่งและการบังคับเพดานการไหลข้ามโซนขนานกันได้ตามแบบ Sealevel

4.7.2 Governance: ชั้นควบคุม PoA บนเชน

โปรแกรม governance ทำหน้าที่เป็นชั้นควบคุม (control plane) แบบ Proof of Authority บนเชน ออกแบบเป็นโปรแกรมเดียวที่รวม 22 instruction โดยมีห้าระบบย่อยหลักตามหน้าที่ (เสริมด้วยคำสั่งเฉพาะ เช่น การเริ่มต้น REC mint การตั้งค่า ZoneConfig และ `get_governance_stats`) จุดสำคัญเชิงสถาปัตยกรรมคือ governance ไม่ได้ทำงานโดดเดี่ยว แต่เป็นแหล่งความจริง (source of truth) ที่โปรแกรมอื่นบนเชนอ่านสถานะไปกำหนดพฤติกรรมของตน กล่าวคือ governance บันทึก “นโยบาย” ขณะที่ trading และ oracle เป็นผู้ “บังคับใช้” นโยบายนั้น ณ ขอบเขตของโปรแกรมตนเอง ระบบย่อยทั้งห้าประกอบด้วย

- **ระบบสิทธิ์ผู้มีอำนาจ (Authority):** `initialize_governance` สร้างบัญชี singleton เริ่มต้น และการโอนสิทธิ์ทำแบบสองขั้นตอน `propose_authority_change` → `approve_authority_change` โดยผู้รับโอนต้องลงนามเอง พร้อม `cancel_authority_change` และเวลาหมดอายุ 48 ชั่วโมง ไม่มีเส้นทางโอนสิทธิ์แบบขั้นตอนเดียว
- **ระบบควบคุมพารามิเตอร์ (Config gates):** `update_governance_config`, `update_erc_limits`, `set_maintenance_mode` (สวิตช์หยุดทั้งระบบ), `update_authority_info` และ `set_oracle_authority` ทุกคำสั่งต้องลงนามโดย authority
- **ระบบใบรับรองพลังงานหมุนเวียน (ERC certificates):** `issue_erc` ตรวจสอบนโยบายใน `GovernanceConfig` แล้วเรียก CPI ไปยัง registry เพื่อทำเครื่องหมายว่าพลังงานถูกอ้างสิทธิ์แล้ว (`mark_erc_claimed`) ร่วมกับ `validate_erc_for_trading`, `transfer_erc` และ `revoke_erc`
- **ระบบลงคะแนน DAO:** `create_proposal` → `cast_vote` (ถ่วงน้ำหนักตามกำลังผลิต) → `execute_proposal` โดยจำกัดให้แก้ไขเฉพาะพารามิเตอร์ของ ZoneConfig ที่กำหนดไว้เท่านั้น ไม่แตะต้องอำนาจ PoA
- **ระบบ allow-list ของ Aggregator:** `admit_aggregator` และ `revoke_aggregator` (เฉพาะ authority) ซึ่งแต่ละรายการคือบัญชี PDA เฉพาะ ทำหน้าที่รับเข้า (admission) โหนด validator นอกเชนที่ละราย

บัญชีสถานะ (state account) ของโปรแกรมเป็นบัญชี PDA ที่กำหนด address ได้แบบ deterministic จาก seed เฉพาะ ดังสรุปในตารางที่ 7

บัญชี (PDA)	Seed	เก็บข้อมูล
GovernanceConfig	[governance_config]	singleton: authority, pending_authority, นโยบาย ERC, maintenance flag, min_quorum_votes และ ตัวนับ
ErcCertificate	[erc_certificate, cert_id]	REC: owner, พลังงาน (kWh), สถานะ, วันหมดอายุ
AggregatorEntry	[aggregator, pubkey]	โหนด validator นอกเซตที่ถูก admit (allow-list)
ZoneConfig	[zone_config, zone_id]	พารามิเตอร์รายโซน: wheeling charge, incentive, loss factor
Proposal	[proposal, zone, id]	ข้อเสนอ DAO: พารามิเตอร์เป้าหมาย, คะแนนเห็นด้วย/ไม่เห็นด้วย, สถานะ, เวลาหมดอายุ
VoteRecord	[vote, proposal, voter]	การลงคะแนนหนึ่งเสียงต่อคู่ (proposal, voter)

ตารางที่ 7: บัญชีสถานะ (PDA) ของโปรแกรม governance: seed และข้อมูลที่เก็บ บัญชี GovernanceConfig เป็น singleton ตรึงขนาดคงที่ 405 ไบต์เพื่อรองรับการอัปเกรด

จุดที่ทำให้ governance มีสถานะเป็น control plane อย่างแท้จริงคือวิธีที่โปรแกรมอื่นบริโภคนโยบายของมัน โปรแกรม trading อ่านบัญชี GovernanceConfig ในทุกคำสั่งสร้างคำสั่งซื้อขาย แล้วเรียก `is_operational()` เพื่อปิดกั้นการทำงานเมื่อระบบอยู่ในโหมดบำรุงรักษา พร้อมตรวจสอบความสมบูรณ์ของ ERC ก่อนรับคำสั่งขาย ขณะที่โปรแกรม oracle ตรวจสอบบัญชี AggregatorEntry เพื่ออนุญาตเฉพาะโหนดที่ถูก admit แล้วให้ส่งค่าอ่านได้ ทั้งสองกรณีเป็นการอ่านสถานะแบบ read-only ด้วยการ deserialize บัญชีเองและตรวจสอบ owner กับ address ของ PDA โดยไม่เรียก CPI ซึ่งหลีกเลี่ยงต้นทุนของ CPI และทำให้การบังคับใช้นโยบายเป็นเพียงการตรวจสอบบัญชีที่ส่งเข้ามาในธุรกรรมนั้น

โครงสร้างนี้สะท้อน PoA สองชั้นที่แยกจากกัน ได้แก่ ชั้นปฏิบัติการ (operational) ที่กำหนดว่าใครได้รับอนุญาตให้รัน validator บนเครือข่าย permissioned และชั้นแอปพลิเคชัน (application-layer) ที่ `GovernanceConfig.authority` ควบคุมสิทธิ์การบริหารโปรแกรม โดย DAO ถูกจำกัดอำนาจ (power-bounded) ให้ปรับได้เฉพาะพารามิเตอร์ของโซน ไม่สามารถยึดอำนาจ authority หรือแก้ไข allow-list ของ validator ได้ นอกจากนี้บัญชี GovernanceConfig ยังถูกตรึงขนาดไว้คงที่ที่ 405 ไบต์พร้อมพื้นที่สำรองสำหรับการอัปเกรด ทำให้เพิ่มฟิลด์ใหม่ได้โดยไม่ต้องย้ายบัญชีเดิม

วงจรชีวิตของใบรับรอง ERC ออกแบบรอบคุณสมบัติห้ามอ้างสิทธิ์ซ้ำ (anti-double-claim) ก่อนออกใบรับรอง คำสั่ง `issue_erc` คำนวณพลังงานที่ยังไม่ถูกอ้างสิทธิ์เป็น $\text{unclaimed} = \text{total_generation} - \text{claimed_erc_generation} - \text{settled_net_generation}$ แบบ saturating แล้วบังคับว่าปริมาณที่ขออนุญาตไม่เกินค่านี้นั้น มิฉะนั้นปฏิเสธ เมื่อผ่านเงื่อนไข โปรแกรมเรียก CPI ไปยัง registry เพื่อเพิ่มตัวนับพลังงานที่ถูกอ้างสิทธิ์ ทำให้พลังงานหน่วยเดียวกันถูกรับรองซ้ำไม่ได้ ใบรับรองที่ออกใหม่มีสถานะ Valid แต่ตั้งค่า `validated_for_trading` เป็นเท็จจนกว่าจะผ่านคำสั่ง `validate_erc_for_trading` ซึ่งเป็นเงื่อนไขบังคับก่อนใบรับรองจะนำไปค้าคำสั่งซื้อขายได้

กลไก maintenance แสดงคุณสมบัติของสวิตช์หยุดทั้งระบบ (system-wide kill switch) ผ่านการเขียนบัญชีจุดเดียว เมื่อ authority เรียก `set_maintenance_mode(true)` ค่า boolean เพียงค่าเดียวบนบัญชี singleton จะพลิก ส่งผลให้ทุกคำสั่งสร้างคำสั่งซื้อขายในโปรแกรม trading ทุกโซนถูกปฏิเสธด้วย MaintenanceMode ทันทีโดยไม่ต้อง deploy โปรแกรม trading ใหม่

4.7.3 Registry: การลงทะเบียนและหลักประกันของ Validator

โปรแกรม registry เป็นทะเบียนกลางของผู้ใช้ มาตราวัด และผู้ตรวจสอบ (validator) คำสั่ง `register_user` และ `register_meter` สร้างบัญชี `UserAccount` และ `MeterAccount` แบบ zero-copy ที่ผูกกับเจ้าของผ่าน PDA นอกจากบทบาททะเบียนแล้ว โปรแกรมนี้ยังถือหลักประกัน (security bond) ของผู้ตรวจสอบ โดย `register_validator` กำหนดให้ต้องวางเงินค้ำ (stake) GRX ขั้นต่ำ 10,000 GRX ก่อนได้รับสถานะ validator ขณะที่ `stake_grx` โอน GRX เข้าคลังกลางและมีช่วงรอถอน (cooldown) 24 ชั่วโมง เมื่อผู้ตรวจสอบประพฤติผิด คำสั่ง `slash_validator` ที่ลงนามโดย authority แบบ PoA จะหักหลักประกันแบบปรับตามความรุนแรง คือ $\text{slash} = \lfloor \text{bond} \times \text{slash_bps} / 10000 \rfloor$ โดยจ่ายชดเชยผู้เสียหายไม่เกินความเสียหายที่พิสูจน์ได้ ($\text{compensation} = \min(\text{slash}, \text{proven_loss})$) และส่วนที่เหลือเข้ากองทุน slash เพื่อตัดแรงจูงใจการกล่าวหาเพื่อหวังรางวัล การหักเต็มจำนวนเปลี่ยนสถานะเป็น Slashed (ถาวร ห้ามลงทะเบียนซ้ำ) ส่วนการหักบางส่วนจนต่ำกว่าขั้นต่ำเปลี่ยนเป็น Suspended (กู้คืนได้ด้วยการเติมเงินค้ำ) กลไก stake-and-slash นี้เป็นแรงจูงใจเชิงเศรษฐศาสตร์ที่เสริมการควบคุมการเข้าร่วมแบบ PoA

4.7.4 Oracle: การรับค่าอ่านแบบขนานและการเคลียร์ตลาด

โปรแกรม oracle รับค่าอ่านมาตราวัดเข้าสู่เซ่นผ่าน `submit_meter_reading` โดยออกแบบให้เขียนลงบัญชี `MeterState` รายมาตราวัด (`seed [meter, meter_id]`) ขณะที่บัญชี `OracleData` ส่วนกลางถูกอ่านอย่างเดียว ค่าอ่านของมาตราวัดต่างตัวจึงมีชุดบัญชีที่เขียน (write set) ไม่ทับซ้อนกันและประมวลผลแบบขนานได้ตามแบบจำลอง Sealevel อย่างไรก็ตามโค้ดระบุข้อจำกัดตามจริงว่า การขนานเกิดขึ้นเมื่อผู้ลงนามจ่ายค่าธรรมเนียม (fee payer) ต่างกัน เท่านั้น หากค่าอ่านหลายรายการใช้ผู้ลงนาม gateway เดียวกัน รายการเหล่านั้นยังถูกประมวลผลแบบลำดับเพราะบัญชีผู้จ่ายถูกล็อกเขียน ก่อนรับค่าอ่าน โปรแกรมตรวจความผิดปกติ (anomaly) ด้วยการตรวจช่วงค่าพลังงานและอัตราส่วนการผลิตต่อการใช้ ซึ่งคำนวณแบบจำนวนเต็มเพื่อเลี่ยงเลขทศนิยม ($\text{produced} \times 100 \leq \text{max_ratio} \times \text{consumed}$) การเคลียร์ตลาดทำผ่าน `trigger_market_clearing` ที่บังคับให้ขอบหน้าต่าง epoch ตรงกับ 900 วินาที ($\text{epoch_timestamp} \% 900 == 0$) และบันทึก `last_cleared_epoch` เพื่อกันการเคลียร์ซ้ำหรือย้อนเวลา

4.7.5 Energy Token: การออกโทเคนแบบ Idempotent และการกำกับด้วย REC

โปรแกรม energy-token จัดการการสร้าง (mint) โทเคนพลังงานภายใต้การกำกับของคณะผู้รับรอง REC (สูงสุด 5 ราย) เส้นทาง `mint_to_wallet` และ `mint_generation` บังคับการร่วมลงนามของผู้รับรอง REC เสมอ (ปฏิเสธเมื่อยังไม่มีผู้รับรอง) โดย `mint_generation` บังคับเพิ่มเป็นแบบ 2-of-2 คือผู้ร่วมลงนาม REC ต้องไม่ใช่ authority เดียวกัน ส่วน `mint_tokens_direct` บังคับการร่วมลงนามเฉพาะเมื่อมีการตั้งผู้รับรองไว้แล้วเท่านั้น จึงเป็นการร่วมลงนามของหน่วยรับรองพลังงานหมุนเวียนก่อนสร้างโทเคน คำสั่ง `mint_generation` ที่สร้างโทเคนจากการผลิตจริงรับประกันความไม่ซ้ำซ้อน (idempotency) ต่อหนึ่งหน้าต่างเวลา ด้วยบัญชี `GenerationMintRecord` รายคู่ (`meter_id, window_start_ms`) ที่สร้างแบบ `init_if_needed` และบังคับให้ขอบหน้าต่างตรงกับ 900,000 มิลลิวินาที โดยตั้งธง `minted` เป็นจริงหลังการ mint สำเร็จเท่านั้น การเรียกซ้ำที่หน้าต่างเดิมจึงคืนค่าเป็น no-op และหากการ mint ล้มเหลว ธงยังเป็นเท็จทำให้ลองใหม่ได้ ส่วนคำสั่ง `retire_energy_tokens` เผาโทเคนผ่าน token interface ของ SPL Token-2022 [33]

4.7.6 Treasury: การบันทึกการชำระและการตรึงค่า THBG

โปรแกรม treasury เป็นปลายทาง CPI ของการบันทึกการชำระจากโปรแกรม trading และเป็นแกนการเงินของระบบ การบันทึกการชำระแบบกลุ่ม (`record_settlement_batch`) เขียนบัญชี `SettlementRecord` รายคู่ (`zone_id, batch_id`) ที่เก็บรากเมอร์เคิล (`merkle_root` ขนาด 32 ไบต์) มุลค่ารวม และภาษีมูลค่าเพิ่ม เป็นข้อผูกพันสำหรับตรวจสอบย้อนหลัง โดยรากเมอร์เคิลผูกชุดธุรกรรมในกลุ่มไวน์เซนขณะทีใบไม้ (leaf) ของต้นไม้ถูกเก็บนอกเซ่น เพื่อรองรับการชำระพร้อมกันจำนวนมากโดยไม่ให้บัญชี treasury กลายเป็นคอขวด จึงมีคำสั่งแบบขนาน

(record_settlement_batch_sharded) ที่เพิ่มยอดลงบัญชีตัวสะสมรายชาร์ตแทนยอดรวมส่วนกลาง โดยแบ่งออกเป็น 16 ชาร์ต (shard_id ที่ผู้เรียกส่งเข้ามาและตรวจช่วง shard_id < 16) แล้วกระทบยอดเข้าสู่ศูนย์กลางภายหลังด้วย aggregate_settlement_shards ด้านการตรึงค่า stablecoin THBG คำสั่ง swap_grx_for_thbg บังคับว่าอุปทานใหม่ต้องไม่เกินทุนสำรองที่ได้รับการรับรอง (new_supply <= attested_reserve) และการรับรองต้องยังไม่หมดอายุ ส่วน redeem_thbg_for_grx จำกัดการไถ่ถอนไม่ให้เกินหลักประกันในคลัง

4.7.7 การประมวลผลแบบขนานด้วย Sealevel และการแบ่งชาร์ต

รูปแบบการออกแบบบัญชีที่ปรากฏซ้ำในหลายโปรแกรมคือการใช้บัญชี PDA รายเอนทิตี เพื่อให้ธุรกรรมที่ไม่เกี่ยวข้องกัน มีชุดบัญชีที่เขียนไม่ทับซ้อนและประมวลผลแบบขนานได้ตามแบบจำลอง Sealevel ของ Solana ตัวอย่างเช่น บัญชี MeterState รายมาตรวัดในโปรแกรม oracle, บัญชี Order และ order nullifier รายคำสั่งในโปรแกรม trading และบัญชี escrow รายผู้ใช้ สำหรับตัวนับรวมที่หากเขียนลงบัญชีเดียวจะกลายเป็นคอขวด (เช่น จำนวนผู้ใช้สะสมหรือยอดชำระสะสม) ระบบใช้การแบ่งชาร์ตจำนวน 16 ชาร์ต โดยโปรแกรม registry เลือกชาร์ตแบบกำหนดได้จากไบนารีแรกของกุญแจ (key.to_bytes())[0] % 16) ส่วนโปรแกรม treasury รับ shard_id จากผู้เรียกและตรวจช่วง (< 16) แล้วจึงกระทบยอดเข้าสู่ตัวนับศูนย์กลางภายหลังด้วยคำสั่งระดับผู้ดูแล (aggregate_shards, aggregate_settlement_shards และ aggregate_readings) ที่ป้องกันการนับซ้ำด้วยบิตแมสก์ ผลคือบัญชีส่วนกลาง เช่น GovernanceConfig, OracleData และ Market ถูกออกแบบให้อ่านแบบล่าช้าได้ (stale) บนเส้นทางที่มีความถี่สูง โดยยอมรับว่าค่ารวมจะตามหลังเล็กน้อยเพื่อแลกกับ throughput ที่ขนานได้

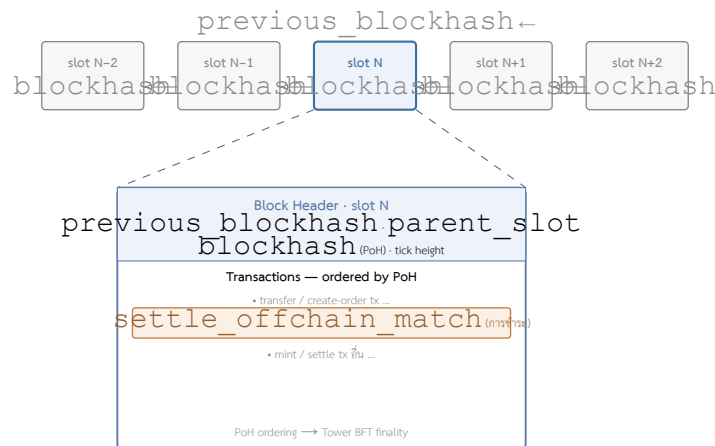
4.8 การชำระธุรกรรมแบบ Atomic Delivery-versus-Payment

คู่คำสั่งที่จับคู่แล้วถูกชำระผ่านคำสั่ง settle_offchain_match ของโปรแกรม trading ซึ่งทำงานแบบส่งมอบพร้อมชำระเงิน (Delivery-versus-Payment: DvP) ภายในธุรกรรมเดียว กล่าวคือการโอนเงินและการโอนพลังงานเกิดขึ้นพร้อมกันหรือไม่เกิดขึ้นเลย หากการโอนใดล้มเหลวธุรกรรมทั้งหมดจะถูกย้อนกลับ จึงไม่มีสถานะค้างที่ฝ่ายหนึ่งได้รับโดยอีกฝ่ายไม่ได้รับ ก่อนการโอน โปรแกรมตรวจลายเซ็น Ed25519 ของทั้งผู้ซื้อและผู้ขายผ่าน instruction sysvar บนข้อความที่ผูกฟิลด์สำคัญของคำสั่ง ได้แก่ order_id, user, energy_amount, price_per_kwh, side, zone_id และ expires_at ทำให้ดัดแปลงปริมาณหรือราคาหลังการลงนามไม่ได้ จากนั้นตรวจเงื่อนไขการครอบราคา คือ $p_s \leq p^* \leq p_b$ ตรวจ side ของทั้งสองฝั่ง และตรวจเวลาหมดอายุ

การชำระเป็นแบบ partial-fill โดยบัญชี order nullifier (seed [nullifier, user, order_id]) เก็บปริมาณที่ถูกเติมสะสมแทนค่าบูตัส คำสั่งจึงถูกเติมได้หลายครั้งจนเต็มปริมาณที่ลงนามไว้ แต่ผลรวมจะไม่เกินปริมาณนั้นทั้งสองฝั่ง สำหรับธุรกรรมข้ามโซนที่ใช้สายส่งระหว่างโซน โปรแกรมยังบังคับเขตแดนการไหลสะสมบนเซน ส่วนธุรกรรมภายในโซนเดียวกันได้รับการยกเว้น การคำนวณทั้งหมดใช้ checked arithmetic แบบจำนวนเต็มปัดลง คือปฏิเสธธุรกรรมเมื่อเกิด overflow หรือเมื่อผลรวมของค่าธรรมเนียม wheeling และ loss เกินมูลค่า V แทนการปัดยอดลงเป็นศูนย์ พร้อมเพดานค่าธรรมเนียมเครือข่ายรวมไม่เกิน 20% ของ V นอกจากนี้โปรแกรมยังมีคำสั่งชำระแบบกลุ่ม (batch_settle_offchain_match) ที่รวมได้สูงสุด 4 คู่ต่อธุรกรรม แต่เพดานเชิงปฏิบัติถูกจำกัดด้วยขนาดแพ็กเก็ตธุรกรรม 1,232 ไบต์ เนื่องจาก payload ของลายเซ็นอยู่ในส่วนข้อมูลคำสั่ง (ราว 189 ไบต์ต่อคำสั่ง) มิใช่ในบัญชี จึงลดขนาดด้วยตาราง address lookup table (ALT) ซึ่งบีบอัดเฉพาะรายการบัญชีไม่ได้ กล่าวคือการรวมสองคู่ (คำสั่งตรวจลายเซ็น 4 คำสั่ง ร่วมกับ BatchMatchPair ที่ serialize แล้ว และรายการดัชนีบัญชีกับ header) ก็เกินขนาดแพ็กเก็ตแล้ว (ค่าไบต์ต่อคำสั่งตรวจลายเซ็นในที่นี้เป็นค่าประเมินจากโครงสร้างคำสั่ง Ed25519 มิใช่ค่าที่วัดในโค้ด) การเพิ่มจำนวนคู่ต่อธุรกรรมจึงต้องเปลี่ยนวิธีบรรจุลายเซ็น เช่น บัญชีลายเซ็นที่ตรวจไว้ล่วงหน้าหรือ multisig แบบรวมนอกเซน มิใช่เพียงเพิ่มจำนวนคู่

เนื่องจากโปรแกรมทำงานบนเครือข่าย Solana-compatible โครงสร้างบล็อกและส่วนหัวบล็อก (block header) จึงเป็นไปตามนิยามของแพลตฟอร์ม มิได้ถูกปรับแต่งในระดับโปรแกรม แต่ละบล็อกผูกกับช่องเวลา (slot) ที่มีเป้าหมายประมาณ 400 มิลลิวินาที ลำดับของธุรกรรมภายในถูกกำหนดด้วยลำดับ PoH ก่อนการลงมติด้วย Tower BFT ส่วนแต่ละธุรกรรมอ้างอิง

blockhash ล่าสุดเพื่อกำหนดอายุและกันการล้นซ้ำในระดับธุรกรรม ห่วงโซ่บล็อกที่เชื่อมต่อกันด้วย previous blockhash และตำแหน่งของคำสั่งชำระภายในบล็อกแสดงในภาพที่ 7



ภาพที่ 7: โครงสร้างบล็อกบนเครือข่าย Solana-compatible (กำหนดโดยแพลตฟอร์ม ไม่ได้ปรับแต่งในระดับโปรแกรม): ห่วงโซ่บล็อกต่อเนื่องที่แต่ละบล็อกอ้างอิงบล็อกก่อนหน้าผ่าน previous blockhash ด้านล่างขยายบล็อก slot N ที่บรรจุคำสั่งชำระ settle_offchain_match

4.9 แบบจำลองข้อมูลและขอบเขตความรับผิดชอบ

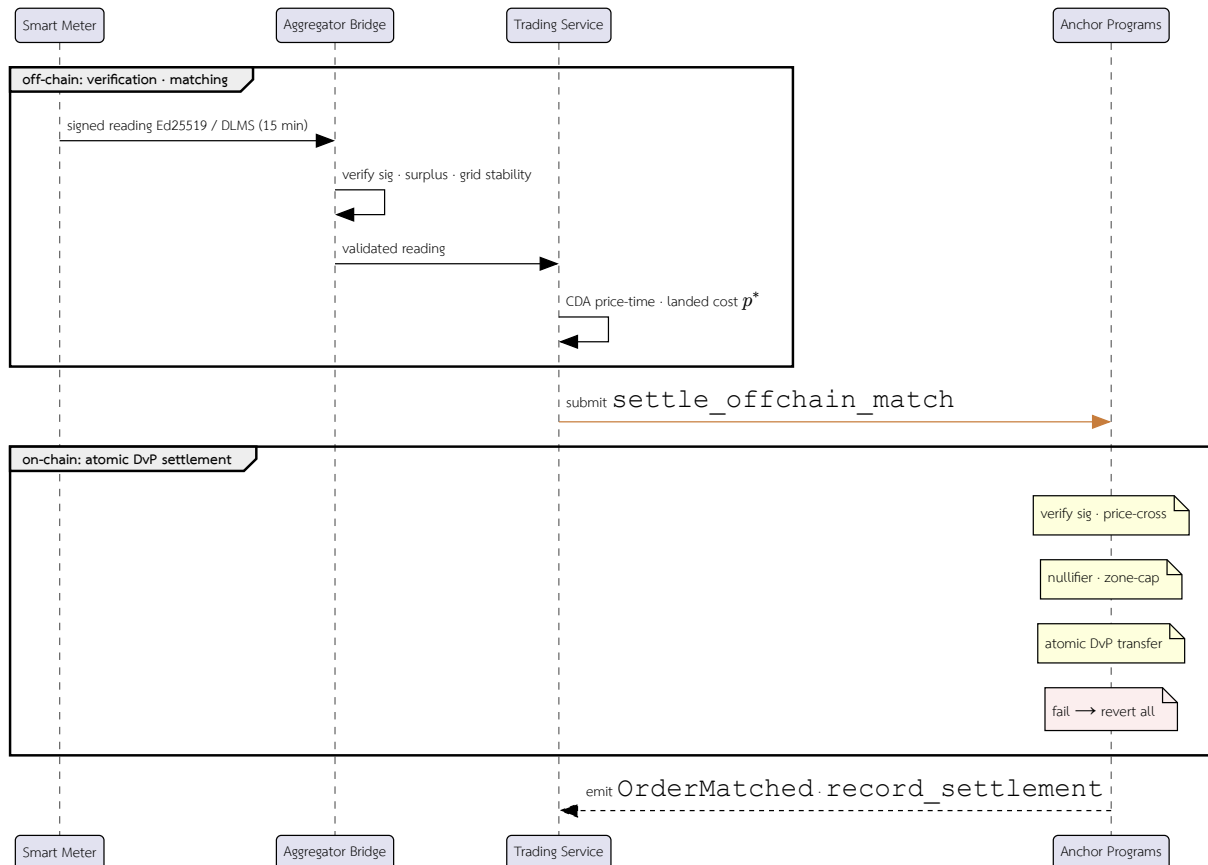
ข้อมูลหลักของคำสั่งซื้อขายประกอบด้วย order_id, user_id (บทบาท prosumer หรือ consumer), meter_id, side (offer หรือ bid), energy_amount (quantity_kwh), price_per_kwh, status, expires_at, epoch_id และ zone_id โดยปริมาณ energy_amount ต้องไม่เกิน available_surplus_kwh ที่ Aggregator Bridge รับรองสำหรับผู้ขายในช่วงเวลาเดียวกัน ซึ่งเป็นเงื่อนไขที่ตรวจสอบในชั้น off-chain ส่วนการป้องกันการส่งซ้ำใช้บัญชี nullifier บนเซตแทนการเก็บ nonce ไว้ในตัวคำสั่ง ส่วน settlement record ประกอบด้วย trade_id, คู่ buy_order_id/sell_order_id, energy_amount ที่ clear, price, fee_amount/net_amount, wheeling_charge, loss_factor, erc_certificate_id, blockchain_tx และ settlement timestamp ทั้งนี้สถานะ escrow ถูกเก็บแยกเป็นบัญชี PDA ต่างหาก ไม่ได้อยู่ในตัว settlement record

ความสามารถในการตรวจสอบย้อนหลัง (auditability) ที่ทำให้บล็อกเชนทำหน้าที่เป็นชั้น audit มาจาก Event log ที่โปรแกรมปล่อยในทุกขั้นตอนสำคัญของวงจรการซื้อขาย ได้แก่ การสร้างคำสั่ง (SellOrderCreated, BuyOrderCreated), การจับคู่และการชำระผ่านเหตุการณ์ OrderMatched ที่บันทึกคู่คำสั่ง ผู้ซื้อ-ผู้ขาย ปริมาณ ราคา มูลค่ารวม และค่าธรรมเนียม, การยกเลิกคำสั่ง (OrderCancelled), การฝากหลักประกัน (EscrowDeposited) และการเคลียร์ตลาด (AuctionCleared) ส่วนการบันทึกการชำระแบบกลุ่มในชั้น treasury ปล่อยเหตุการณ์ SettlementBatchRecorded ที่ผู้กรากเมอร์เคลของกลุ่มไว้บนเซต เหตุการณ์เหล่านี้เป็นบันทึกที่ไม่ถูกแก้ไขย้อนหลัง (append-only) ซึ่งผู้ตรวจสอบภายนอกใช้ประกอบประวัติธุรกรรมที่ตรวจทานได้โดยไม่ต้องเชื่อถือชั้น off-chain โดยตรง

ขอบเขตความรับผิดชอบถูกกำหนดให้ชัดเจนดังนี้ Backend และ Aggregator Bridge เป็นผู้ประเมินข้อมูลกำลังผลิตและการใช้ไฟฟ้า เงื่อนไข Grid stability, Islanding safety, ข้อจำกัดของโครงข่าย และเงื่อนไขปริมาณ kWh ที่ไม่เกินค่ารับรอง (available_surplus) จากนั้นจึงปล่อยให้คู่คำสั่งที่ผ่านเงื่อนไขถูกส่งไปยังขั้นตอน settlement บนเซต ส่วน Anchor program ตรวจสอบเฉพาะ account ownership, signer, ลายเซ็น Ed25519 ของทั้งผู้ซื้อและผู้ขายบน order payload, timestamp validity, การกันส่งซ้ำผ่านบัญชี order nullifier, เงื่อนไข slippage/zone-capacity, สถานะ escrow และสถานะ order/trade ที่ยังไม่ถูกใช้ซ้ำ ดังนั้นบล็อกเชนในต้นแบบนี้ทำหน้าที่เป็น settlement and audit layer ไม่ใช่ตัวคำนวณ power-flow หรือ grid-stability engine แบบเต็มรูปแบบ

4.10 วงจรการซื้อขายทั้งระบบ

ลำดับการซื้อขายถูกแบ่งขอบเขตการทำงานระหว่าง off-chain และ on-chain อย่างชัดเจน ดังภาพที่ 8 ฝั่ง off-chain รับผิดชอบการรวบรวมข้อมูล Smart Meter การยืนยันตัวตน การตรวจสอบเวลาอ้างอิง การประเมินเงื่อนไข Grid stability และการคำนวณค่าสิ่งที่เสนอให้ clear ส่วนฝั่ง on-chain รับผิดชอบเฉพาะการยืนยัน account state, ลายเซ็น Ed25519 ของผู้ซื้อและผู้ขายบน order payload ที่ลงนามไว้, bounded matched pair, escrow และการบันทึก settlement event



ภาพที่ 8: วงจรการซื้อขายในรูปแบบ sequence diagram: ฝั่ง off-chain (Smart Meter → Aggregator Bridge → Trading Service) ตรวจสอบและจับคู่ แล้วส่งให้โปรแกรม Anchor บนเชน (ผ่าน Chain Bridge) ชำระและบันทึกหลักฐาน เหตุการณ์ระบุเงื่อนไขที่บังคับใช้บนเชน หากขั้นตอนใดล้มเหลวธุรกรรมทั้งหมดถูกย้อนกลับ

กระบวนการนี้ช่วยให้ธุรกรรมพลังงานมีความโปร่งใสและตรวจสอบย้อนหลังได้ พร้อมลดความเสี่ยงจากการนำข้อมูล Smart Meter ที่ยังไม่ผ่านการยืนยันเข้าสู่บล็อกเชนโดยตรง การแยก boundary ระหว่าง off-chain verification และ on-chain settlement จึงเป็นสำคัญในการรักษาทั้งประสิทธิภาพของระบบและความปลอดภัยของไมโครกริด

5. ผลการทดลอง

5.1 การตั้งค่าการทดลองและภาระงาน

การทดลองทั้งหมดรันบนเครื่อง Apple M2 (8 cores, หน่วยความจำ 16 GiB) โดยระบบจัดเรียงเป็น superproject ที่รวมแต่ละบริการเป็น git submodule ทำให้ระบุเวอร์ชันของสิ่งประดิษฐ์ได้จาก commit ของ superproject ร่วมกับ pointer ของแต่ละ submodule ภาระงานหลักกำหนดด้วย Smart Meter 80 เครื่องที่ส่งค่าอ่านทุก 15 วินาทีของเวลาจำลอง ซึ่งสอดคล้องกับการบีบอัดเวลาประมาณ 60 เท่าเมื่อเทียบกับหน้าต่างเวลา 15 นาที (900 วินาที) ของรอบการส่งข้อมูลของมาตรวัดจริง ในที่นี้นิยม “ค่าอ่าน” ที่ใช้เป็นหน่วยวัดปริมาณงานว่าหมายถึงค่าอ่านมาตรวัดหนึ่งรายการที่ลงนามด้วย Ed25519 แล้วผ่านการตรวจสอบลายเซ็นและถูกกระจายเข้าสู่ Redis Stream ที่ Aggregator Bridge มีใช้คำสั่งซื้อขายที่จับคู่แล้วหรือธุรกรรม settlement บนบล็อกเชน ตัวแปรหลักสรุปไว้ในตารางที่ 8

พารามิเตอร์	ค่า	ความหมาย
NUM_METERS	80	จำนวน Smart Meter ในการจำลอง
SIMULATION_INTERVAL	15 s	รอบการส่งค่าอ่านในเวลาจำลอง
Time compression	$\approx 60 \times$	เทียบกับหน้าต่างจริง 900 s
Nominal ingest rate	5.33 readings/s	$80 \div 15$ ในเวลาจำลอง
Run duration	≈ 27 min	เวลานานฟิสิกส์ของการรันหนึ่งรอบ
Total readings	26,240	ตรวจลายเซ็นสำเร็จ ไม่มีการสูญหาย

ตารางที่ 8: ตัวแปรของภาระงานสำหรับการประเมินเส้นทางรับข้อมูล

5.2 อัตราการรับข้อมูลของเส้นทาง telemetry ingest

จากการรันจริงหนึ่งรอบเป็นเวลาประมาณ 27 นาที Aggregator Bridge รับและตรวจสอบลายเซ็นค่าอ่านได้ทั้งสิ้น 26,240 รายการจากมาตรวัด 80 เครื่องอย่างต่อเนื่องโดยไม่มีการสูญหายของข้อมูล โดยอัตราที่วัดได้จากเวลานาฬิกาจริงอยู่ที่ประมาณ 16 รายการต่อวินาที ซึ่งสูงกว่าอัตราเชิงออกแบบเนื่องจากรอบการส่งข้อมูลของ simulator ถูกเร่งให้เร็วกว่าช่วงเวลา 15 วินาทีของเวลาจำลอง

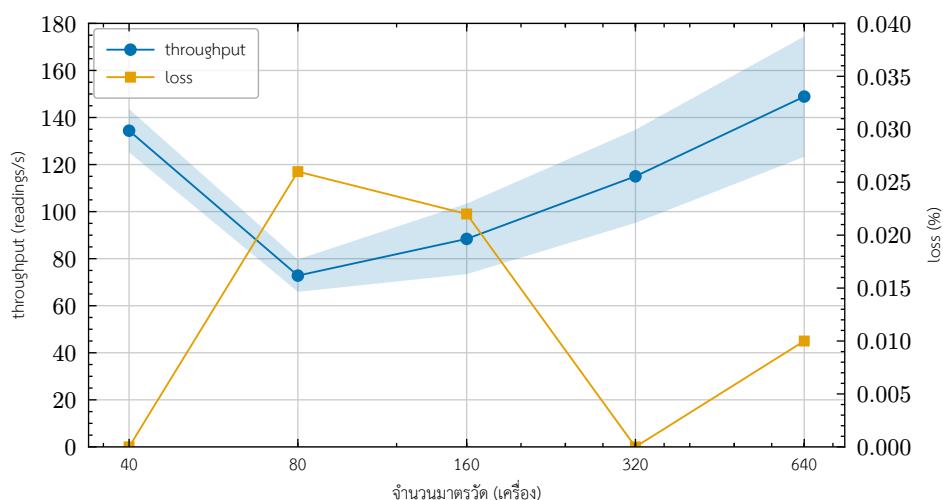
ผลนี้แสดงให้เห็นว่าเส้นทางรับข้อมูล (ingest path) สามารถรองรับภาระการส่งข้อมูลของมาตรวัด 80 เครื่องได้อย่างเสถียร อย่างไรก็ตาม การวัดนี้ครอบคลุมเฉพาะเส้นทางรับและตรวจสอบข้อมูลมาตรวัดเท่านั้น ยังไม่รวมการวัดปริมาณงานและความหน่วงของเส้นทาง settlement บนเซน

5.3 ขอบเขตการขยายตัวและสัดส่วนการสูญหายภายใต้ภาระงาน

เพื่อประเมินขอบเขตความสามารถให้กว้างกว่าอัตราเชิงออกแบบ 5.33 รายการต่อวินาที ผู้จัดทำเพิ่มภาระงานแบบขั้นบันไดด้วยจำนวนมาตรวัด 40, 80, 160, 320 และ 640 เครื่อง โดยแต่ละขั้นรันเป็นเวลา 60 วินาที และทำซ้ำ 5 รอบเพื่อรายงานค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน ในแต่ละรอบ simulator ส่งค่าอ่านที่ลงนาม Ed25519 แบบต่อเนื่องสูงสุดโดยไม่มีการหน่วงระหว่างรอบส่ง ผลสรุปไว้ในตารางที่ 9 และแสดงแนวโน้มเป็นกราฟในภาพที่ 9

มาตรวัด (เครื่อง)	throughput (readings/s)	loss (สูงสุด)
40	134.4 ± 9.2	0
80	72.8 ± 6.9	2.6×10^{-4}
160	88.4 ± 15.0	2.2×10^{-4}
320	115.0 ± 19.8	0
640	148.9 ± 25.6	1.0×10^{-4}

ตารางที่ 9: อัตราการรับข้อมูลและสัดส่วนการสูญหายเทียบกับขนาดฟลิตมาตรวัด (ค่าเฉลี่ย $\pm$ ส่วนเบี่ยงเบนมาตรฐาน จาก 5 รอบ รอบละ 60 วินาที)



ภาพที่ 9: อัตราการรับข้อมูล (แกนซ้าย) และสัดส่วนการสูญหายของค่าขอ (แกนขวา) เทียบกับขนาดฟลิตมาตรวัด

ผลการวัดมีสองข้อสังเกตหลัก ข้อแรกคือสัดส่วนการสูญหายของข้อมูลอยู่ในระดับใกล้เคียงศูนย์ตลอดทุกขนาดพล็อต โดยค่าสูงสุดที่วัดได้คือประมาณ 2.6×10^{-4} (ประมาณ 0.03%) ที่ขนาด 80 เครื่อง และหลายรอบไม่มีการสูญหายเลย ข้อที่สองคืออัตราการรับข้อมูลที่วัดได้ไม่เพิ่มขึ้นแบบเอกฐานตามจำนวนมาตรวัด แต่แกว่งอยู่ในช่วงประมาณ 73–149 รายการต่อวินาที เนื่องจากการทดลองนี้สร้างภาระงานจากไคลเอนต์ผู้ส่งเพียงตัวเดียวที่ส่งค่าอ่านรายการมาตรวัดต่อรอบตามลำดับ ความแกว่งนี้จึงน่าจะสะท้อนข้อจำกัดฝั่งผู้ส่งมากกว่าการอิ่มตัวของบริการรับข้อมูล ตัวเลขที่รายงานจึงเป็น lower bound มิใช่เพดานความสามารถที่แท้จริงของเส้นทางรับข้อมูล

5.4 อัตราการจับคู่คำสั่งในชั้น off-chain

ผู้จัดทำวัดต้นทุนการจับคู่คำสั่งของเครื่องจับคู่ CDA ด้วย micro-benchmark (Criterion) ที่เรียกฟังก์ชันจับคู่หนึ่งรอบเหนือสมุดคำสั่งซื้อ 1,000 รายการและคำสั่งขาย 1,000 รายการ โดยกำหนดให้ราคาคำสั่งซื้อสูงกว่าคำสั่งขายทุกคู่ จึงจับคู่ได้เต็มประมาณ 1,000 คู่ต่อรอบ เก็บ 100 ตัวอย่างหลังช่วง warm-up ผลสรุปในตารางที่ 10

ตัวชี้วัด	ค่า
เวลาต่อรอบจับคู่ 1,000 × 1,000 (median)	32.56 ms
ช่วงความเชื่อมั่น 95%	32.28–32.75 ms
อัตราประมวลผลคำสั่ง (≈)	6.1×10^4 orders/s
อัตราจับคู่คำสั่ง (≈)	3.1×10^4 pairs/s

ตารางที่ 10: อัตราการจับคู่ของเครื่องจับคู่ CDA ในหน่วยความจำ

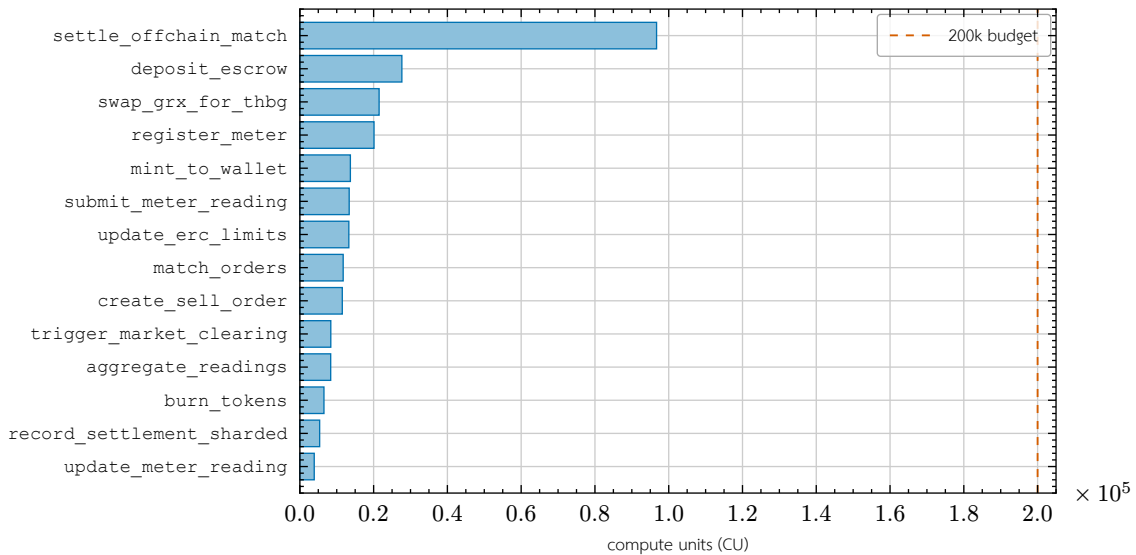
ค่านีวัดเฉพาะการจับคู่ในหน่วยความจำ ไม่รวมต้นทุนการชำระธุรกรรมบนเชน การคงสภาพข้อมูล หรือการสื่อสารผ่านเครือข่าย จึงเป็นขอบเขตบน (upper bound) ของอัตราการจับคู่ที่แยกออกจากต้นทุน settlement อย่างชัดเจน ผลนี้บ่งชี้ว่าในสถาปัตยกรรมแบบแยกชั้น การจับคู่ในชั้น off-chain มิใช่คอขวดของระบบเมื่อเทียบกับต้นทุนการชำระธุรกรรมบนเชน

5.5 ต้นทุนการชำระธุรกรรมบนเชน

ผู้จัดทำวัดต้นทุนการประมวลผลบนเชนของการชำระธุรกรรมโดยตรง โดยรายงานหน่วยประมวลผล (compute units, CU) ที่คำสั่งชำระใช้จริง อ่านจากค่า computeUnitsConsumed ของธุรกรรมที่ยืนยันแล้วบน solana-test-validator 3.1.10 (Agave client) ค่านีเป็นตัวชี้วัดต้นทุนบนเชนที่กำหนดได้แน่นอน (deterministic) และไม่ขึ้นกับฮาร์ดแวร์ของ validator ต่างจากความหน่วงบนเครือข่ายทดสอบเฉพาะที่ ผลการวัดสรุปในตารางที่ 11 และผลต่อ instruction ของโปรแกรมบนเชนทุกตัวแสดงในภาพที่ 10

คำสั่ง (instruction)	compute units
settlement ต่อ 1 คู่คำสั่ง (settle offchain match)	96,707
เส้นทางที่มีขาแลกเปลี่ยน classic-SPL → Token-2022	≈ 103,000
การชำระแบบกลุ่มที่ใช้ Token-2022 ทั้งสองขา	≈ 80,000–92,000

ตารางที่ 11: ต้นทุน compute unit ที่วัดได้ของคำสั่งชำระ (ต่อคู่คำสั่งที่จับคู่แล้ว 1 คู่) ค่า 96,707 CU เป็นค่าของรูปแบบ escrow settlement ที่วัด มิใช่ค่าคงที่สากล



ภาพที่ 10: ต้นทุน compute unit ต่อ instruction ของโปรแกรมบนเซน เทียบกับงบประมาณตั้งต้น 200,000 CU (เส้นประ)

ค่าที่วัดได้คือ 96,707 CU ต่อการชำระหนึ่งคู่คำสั่ง เทียบกับงบประมาณ compute ตั้งต้นต่อคำสั่งที่ 200,000 CU และเพดานสูงสุดต่อธุรกรรมที่ 1,400,000 CU บ่งชี้ว่าการชำระแต่ละครั้งใช้ทรัพยากรประมาณ 48% ของงบตั้งต้น อนึ่ง แม้เพดาน compute สูงสุดต่อธุรกรรมจะรองรับได้ในเชิงทฤษฎีราว 14 คู่ต่อธุรกรรม แต่ข้อจำกัดที่บังคับจริงมิใช่ compute แต่เป็นขนาดแพ็กเก็ตธุรกรรม 1,232 ไบต์ ดังที่อธิบายไว้ในหัวข้อ 3.4 จากภาพที่ 10 ยังเห็นได้ว่าทุก instruction ยกเว้นเส้นทางชำระที่ตรวจสอบลายเซ็นใช้ compute ไม่เกินประมาณ 28k CU หรือราว 14% ของงบตั้งต้น โดยเส้นทางที่มีความถี่สูงอยู่ในระดับต่ำสุด ได้แก่ update_meter_reading (3.9k CU) และ record_settlement_sharded (5.4k CU) ผลนี้ยืนยันด้วยการวัดจริง มิใช่การคาดการณ์ ว่างานประมวลผลหนักถูกย้ายไปทำในชั้น Backend ก่อน ขณะที่ Smart Contract บังคับใช้เฉพาะเงื่อนไขขั้นต่ำที่ตรวจสอบได้บนเซน จึงทำหน้าที่เป็นชั้น settlement และ audit ที่ใช้ compute ต่ำตามที่ออกแบบไว้

5.6 ปริมาณงานธุรกรรมบนเซน

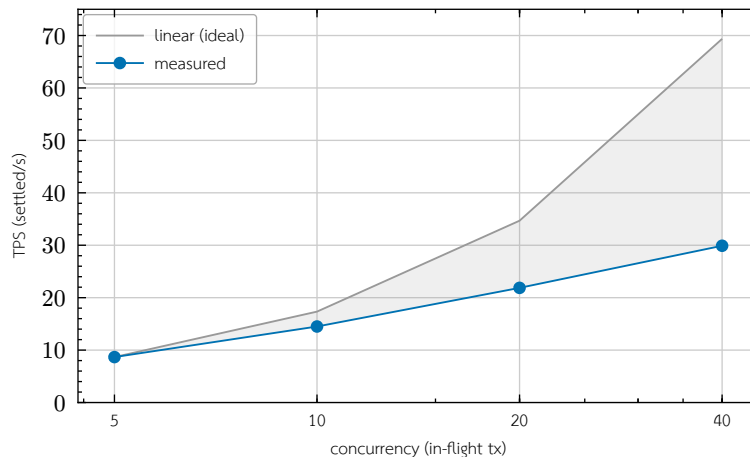
นอกเหนือจากต้นทุน compute ต่อคำสั่ง ผู้จัดทำวัดปริมาณงานของการประมวลผลธุรกรรมบนเซนด้วยชุดภาระงาน OLTP มาตรฐานที่ฟอร์ตมาสู่ account model ของ Solana ได้แก่ BlockBench, SmallBank และ TPC-C เสริมด้วยการวัดเส้นทางการชำระจริง ทั้งหมดรันบน solana-test-validator เดียว (single-node; เครือข่าย permissioned ที่กำกับการเข้าร่วมแบบ PoA) บนเครื่อง Apple M2 โดยความหวังเป็นเวลานานหิกาจริงแบบ client→confirmed และหน่วย compute อ่านจาก computeUnitsConsumed ของธุรกรรมที่ยืนยันแล้ว ผลภาระงาน OLTP แบบลำดับสรุปในตารางที่ 12 และผลของการกวาดค่าระดับการทำงานพร้อมกัน (concurrency sweep) ของ TPC-C สรุปในตารางที่ 13

ภาระงาน (workload · op)	mean ms	TPS	CU/tx
BlockBench · do nothing	719.95	1.39	648
BlockBench · cpu heavy sort	590.83	1.69	9,645
BlockBench · ycsb read (RPC)	4.29	233.18	—
SmallBank · SendPayment	604.63	1.65	5,963
SmallBank · Amalgamate	631.62	1.58	5,936

ตารางที่ 12: ภาระงาน OLTP มาตรฐานที่ฟอร์ตมาสู่ account model (แบบลำดับ, n = 150) ค่า TPS ในตารางนี้ถูกจำกัดด้วยความหวังของรูปแบบโคลเอนต์เดียว มิใช่เพดาน throughput ส่วน ycsb_read เป็นการอ่านบัญชีผ่าน RPC ที่ไม่มีรอบฉันทมติ

concurrency	TPS	mean ms	p95 ms	CU/tx (mean)
5	8.67	575.00	726.10	21,263
10	14.50	679.34	879.95	20,633
20	21.87	856.74	1,656.15	21,269
40	29.90	1,057.79	1,707.00	21,508

ตารางที่ 13: การกวาดค่าระดับการทำงานพร้อมกันของ TPC-C (TX = 500, NewOrder 50% / Payment 50%) สำเร็จ 100% ทุกระดับ



ภาพที่ 11: ปริมาณงาน TPC-C เทียบกับระดับการทำงานพร้อมกันบน validator เดียว: TPS ที่วัดได้ขยายตัวแบบ sublinear (concurrency เพิ่ม 8 เท่า ให้ TPS เพิ่มขึ้นเพียง 3.45 เท่า) โดยมีจุดอิ่มตัวระหว่าง concurrency 10 ถึง 20 ตัวเลขที่สำคัญที่สุดต่อระบบนี้คือปริมาณงานของเส้นทางการชำระจริง มีใช้ของ proxy ทั่วไป เมื่อวัดเส้นทาง batch_settle_offchain_match แบบ open-loop submission sweep ได้อัตราเพียงราว 0.5 TPS และคงที่ทุกระดับ concurrency (conc 5 $\rightarrow$ 0.51, conc 10 $\rightarrow$ 0.58 TPS; goodput 100%, ไม่มี revert บนเชน) แม้กระจายการชำระข้ามทั้ง 16 ชาร์ตก็ได้ตัวเลขใกล้เคียง (0.57–0.59 TPS) เพราะคอขวดมีใช้ชาร์ต แต่เป็นชุดบัญชีส่วนกลางที่ทุกการชำระต้องเขียนเสมอ ได้แก่ บัญชีสะสมยอด total_settled_thbg และบัญชีผู้เก็บค่าธรรมเนียม/ค่าผ่านสาย/การสูญเสีย การชำระจึงถูกผูกกับการเขียนส่วนกลาง (global-write-bound) โดยการออกแบบ ขณะที่หน่วย compute ต่อธุรกรรมใน TPC-C sweep คงที่ราว 21,000 CU ทุกระดับ concurrency ซึ่งคอขวดคือเวลาในการผลิตบล็อก (ราว 400–600 มิลลิวินาที) ร่วมกับการ serialize การเขียนบัญชีส่วนกลาง มีใช้การประมวลผลบนเชน เมื่อเทียบกับหน้าต่างการเคลียร์ตลาด 900 วินาที อัตรา 0.5 TPS ยังรองรับได้ราว 450 การชำระต่อหน้าต่าง ซึ่งเกินความต้องการของภาระงาน 80 มาตรวัดในการประเมินนี้มาก ส่วนเพดาน proxy ที่ 29.9 TPS เทียบเท่าราว 2.69×10^4 ธุรกรรมต่อหน้าต่าง การกวาดค่ายังแสดงการขยายแบบ sublinear (concurrency 5 $\rightarrow$ 40 เพิ่มขึ้น 8 เท่า ให้ TPS เพิ่มขึ้นเพียง 3.45 เท่า) และจุดอิ่มตัว (saturation knee) ระหว่าง concurrency 10 ถึง 20 ที่ส่วนเบี่ยงเบนของความหน่วงและ p95 เพิ่มขึ้นชัดเจน ดังแสดงเทียบกับเส้นอ้างอิงเชิงเส้นในภาพที่ 11

อย่างไรก็ตามผลทั้งหมดมาจาก validator เดียว จึงควรตีความภายใต้ข้อจำกัดสี่ประการ ประการแรก การวัดบน validator เดียวไม่สะท้อนต้นทุน consensus (PoH ร่วมกับ Tower BFT) ของเครือข่าย permissioned หลายโหนด ประการที่สอง BlockBench, SmallBank และ TPC-C เป็น proxy ทั่วไปมิใช่ภาระงานพลังงาน อัตรา TPS ของเส้นทางการชำระจริงจึงต่ำกว่าตัวเลข proxy อย่างมีนัย ประการที่สาม เส้นทางแบบลำดับในตารางที่ 12 ที่ราว 1.4–1.7 TPS เป็น latency-bound มีใช้เพดาน throughput และประการที่สี่ ตัวเลข TPS ของการกวาด TPC-C เป็นค่าจุดเดียวต่อระดับ concurrency โดยรายงานช่วงความเชื่อมั่นเฉพาะของความหน่วงเท่านั้น

5.7 ปริมาณงานของตัวแก้สมการที่ระดับผลิตขนาดใหญ่

การทดลองนี้ขับเคลื่อน `SimulationEngine.tick()` โดยตรง (ข้ามเส้นทางเครือข่ายทั้งหมด รวมถึง Aggregator Bridge และบล็อกเชน) เพื่อแยกวัดเฉพาะต้นทุนของการสร้างค่าอ่านอุปกรณ์ (device-model generation) และการแก้สมการ power-flow บนโครงข่ายอ้างอิง 80 บัส ที่ขนาดผลิต 10,000, 50,000 และ 100,000 มาตรการวัด โดยกำหนดสัดส่วนมาตรการวัดที่มีแผง PV แบบสุ่มไว้ที่ 10% ผลสรุปในตารางที่ 14

จำนวนมาตรการวัด	สัดส่วน PV	median tick	p95 tick	max tick
10,000	9.90%	16.5 s	17.7 s	17.7 s
50,000	9.81%	97.6 s	107.8 s	107.8 s
100,000	9.97%	230.4 s	397.5 s	397.5 s

ตารางที่ 14: ต้นทุนเวลานาฬิกาจริงต่อรอบของตัวแก้สมการเทียบกับขนาดผลิต (การรันสำรวจรอบเดียว, 4 tick ต่อกรณี, ไม่ส่งออกเครือข่าย)

สัดส่วนมาตรการวัดที่มี PV ลู่เข้าสู่ค่าเป้าหมาย 10% เมื่อขนาดผลิตเพิ่มขึ้น สอดคล้องกับทฤษฎีจำนวนมาก (law of large numbers) ของการสุ่มเลือกประเภทมาตรการวัดต่อราย เวลาต่อรอบ (tick) ขยายตัวแบบเหนือเชิงเส้นเล็กน้อยเมื่อเทียบกับขนาดผลิต (100,000 มาตรการวัดใช้เวลาประมาณ 14 เท่าของ 10,000 มาตรการวัด สำหรับผลิตที่ใหญ่กว่า 10 เท่า) ซึ่งบ่งชี้ว่าคอขวดอยู่ที่การสร้างค่าอ่านต่อมาตรการวัด (ทำงานแบบ CPU-bound บนเทรตเดียว) มิใช่การแก้สมการ power-flow ที่มีขนาดโครงข่ายคงที่ไม่ขึ้นกับจำนวนมาตรการวัด

5.8 การยืนยันการ mint บนเชนจริง

การทดลองสุดท้ายเปิดเส้นทางเครือข่ายเต็มรูปแบบ ตั้งแต่การลงทะเบียนเจ้าของมาตรการวัดผ่าน IAM Service การลงทะเบียนกุญแจ Ed25519 ของแต่ละมาตรการวัด การส่งค่าอ่านแบบเข้ารหัส AES-256-GCM ผ่าน mTLS ที่ลงนามแล้ว ไปจนถึงธุรกรรม mint บนเชนจริง ผลสรุปในตารางที่ 15

ผลิต (เครื่อง)	Minted	Overall TPS	Peak TPS (10 s)
100	140 mint / 20 มาตรการวัด	—	—
1,000	1,000 / 1,000	29.3	—
10,000	10,000 / 10,000	18.4	123.2
25,000 (ปรับแต่งแล้ว)	25,000 / 25,000	23.2	193.4

ตารางที่ 15: ปริมาณงาน mint บนเชนจริงที่ระดับผลิตต่าง ๆ (การรันสำรวจเพียงรอบเดียว)

การรัน 100 มาตรการวัด 5 รอบยืนยันว่าทุกขั้นตอนของเส้นทางทำงานสอดคล้องกันแบบ end-to-end กล่าวคือเกิดธุรกรรม mint บนเชนจริง 140 รายการครอบคลุม 20 มาตรการวัดที่ไม่ซ้ำกัน แต่ละรายการมีลายเซ็นธุรกรรม Solana และหมายเลข slot ที่ตรวจสอบได้จริง จึงพิสูจน์ว่าตัวเลขไฟฟ้าส่วนเกินสุทธิที่ขุดจำลองคำนวณสอดคล้องกับสิ่งที่ถูก mint จริงบนเชน สำหรับผลิตขนาดใหญ่ ที่ค่าตั้งต้นก่อนปรับแต่ง การส่งผลิต 25,000 มาตรการวัดในครั้งเดียวทำให้คิวของ mint consumer ที่ Chain Bridge รับภาระเกิน เกิดเป็นวงจรป้อนกลับเชิงบวกที่ goodput ลดลงจาก 16.7 เหลือ 3.7 mint ต่อวินาที การปรับสามจุด ได้แก่ เพิ่ม concurrency ของ mint consumer (8 → 64) ข้ามขั้นตอนจำลองก่อนส่งจริงที่ซ้ำซ้อน และขยายวงเวลารอการตอบกลับของ aggregator (30 → 120 วินาที) ขจัดปัญหานี้ได้ทั้งหมด ทั้งนี้ผลในตารางที่ 15 เป็นการรันสำรวจเพียงรอบเดียว จึงควรตีความเป็นค่าบ่งชี้เชิงออกแบบที่รอการวัดซ้ำ

6. อภิปรายผลและข้อจำกัด

6.1 อภิปรายผล

ผลการออกแบบระบบจำลองชี้ให้เห็นว่าแนวทางการแยกชิ้นการทำงานระหว่าง Smart Meter Simulator, Backend Service และ Solana Smart Contract ช่วยให้ระบบซื้อขายพลังงานแบบ Peer-to-Peer มีโครงสร้างที่ตรวจสอบได้ และมีจุดขยายระบบที่ชัดเจนขึ้น Backend Service ทำหน้าที่เป็นชั้นควบคุมหลักสำหรับตรวจสอบข้อมูล การจัดการสิทธิ์

และการประเมินเงื่อนไขด้าน Grid stability ก่อนส่งธุรกรรมไปยังบล็อกเชน ทำให้ Smart Contract ไม่ต้องรับภาระการประมวลผลทั้งหมดของระบบไมโครกริด อย่างไรก็ตาม ระบบยังต้องให้ความสำคัญกับความปลอดภัยของโครงข่ายไฟฟ้าเป็นลำดับแรก การอนุมัติธุรกรรมจึงไม่ควรพิจารณาเฉพาะราคาและปริมาณพลังงานเท่านั้น แต่ต้องพิจารณาเงื่อนไขด้านเสถียรภาพของระบบและสถานะการเชื่อมต่อของอุปกรณ์ร่วมด้วย

ผลการประเมินสนับสนุนสมมติฐานการออกแบบที่ให้บล็อกเชนเป็นชั้น settlement แบบบาง (thin settlement layer) ในสามประเด็น ประการแรก เส้นทางรับข้อมูลรักษาสัดส่วนการสูญหายให้ใกล้เคียง ($\leq 0.03\%$) จนถึงภาระงาน 640 เครื่อง บ่งชี้ว่าการย้ายงานตรวจสอบและรวบรวมข้อมูลที่มีความถี่สูงไว้ในชั้น off-chain ไม่ได้กลายเป็นคอขวดของระบบภายใต้ภาระระดับที่ทดสอบ ประการที่สอง ต้นทุนการชำระธุรกรรมบนเชนที่วัดได้ 96,707 CU ต่อคำสั่ง คิดเป็นประมาณ 48% ของ compute เริ่มต้นต่อคำสั่ง แสดงว่าเมื่อเกิดกาเชงธุรกิจส่วนใหญ่ถูกบังคับใช้นอกเชนแล้ว ภาระที่เหลือบนเชนอยู่ในระดับที่เปิดช่องให้ compute ให้รวมหลายคำสั่งในธุรกรรมเดียว (batch settlement) ได้ แม้การรวมจริงจะถูกจำกัดด้วยขนาดแพ็คเกจธุรกรรมก่อนเพดาน compute ประการที่สาม การที่ต้นทุนบนเชนเป็นค่าที่กำหนดได้แน่นอน (deterministic CU) มากกว่าจะผันแปรตามค่าธรรมเนียมตลาดแบบเครือข่ายสาธารณะ สอดคล้องกับเหตุผลเชิงการกำกับดูแลในการเลือกเครือข่าย consortium แบบ PoA ที่ต้นทุนธุรกรรมคาดการณ์ได้และไม่ขึ้นกับความผันผวนของค่า gas

ในเชิงกลไกเศรษฐศาสตร์ การแยกบทบาทของโทเคนสามประเภท คือ โทเคนพลังงานที่รับรองด้วย REC สำหรับการส่งมอบเหรียญ THBG ที่ตรงค่าเงินบาทสำหรับการตั้งราคาและชำระเงิน และโทเคน GRX สำหรับการ stake และการกำกับดูแลช่วยให้การชำระธุรกรรมเป็นแบบ DvP ที่ผูกการส่งมอบพลังงานเข้ากับการชำระเงินในธุรกรรมเดียว และให้สิทธิ์การกำกับดูแลเครือข่ายอยู่ภายใต้กรอบ consortium ที่ควบคุมการรับรอง REC และ allow-list ของ aggregator ได้อย่างชัดเจน

เมื่อเทียบกับแพลตฟอร์ม permissioned ที่ใช้แพร่หลายในวรรณกรรมอย่าง Hyperledger Fabric [13] การออกแบบในงานนี้ต่างกันเชิงสถาปัตยกรรมในสามจุด ประการแรก Fabric ใช้แบบจำลองการประมวลผลแบบ execute-order-validate ที่ chaincode รับบน endorsing peers ก่อนเรียงลำดับผ่าน orderer ขณะที่งานนี้ใช้แบบจำลอง account ของ Solana ที่ประมวลผลธุรกรรมแบบขนานตาม Sealevel เมื่อชุดบัญชีที่เขียนไม่ทับซ้อนกัน ซึ่งเอื้อต่อการขยายการส่งคำสั่งรายเอนทิตี แต่ทำให้การชำระที่กระหนาบยอดส่วนกลางถูกผูกกับการเขียนส่วนกลาง (global-write-bound) ดังที่วัดได้ในหัวข้อ 5.6 ประการที่สอง ต้นทุนการประมวลผลในงานนี้แสดงเป็นหน่วย compute (CU) ที่กำหนดได้แน่นอนและมีเพดานชัดเจนต่อธุรกรรมต่างจาก Fabric ที่ไม่มีแนวคิดงบประมาณ compute ต่อธุรกรรมแบบเดียวกัน ทำให้ต้นทุนบนเชนของงานนี้คาดการณ์และตรวจจับการถดถอย (regression) ได้ตรงไปตรงมา ประการที่สาม งานนี้บังคับการตรวจที่มาของค่าอ่านมาตรวัดด้วยลายเซ็น Ed25519 และการกันส่งซ้ำผ่าน order nullifier ภายในตรรกะของโปรแกรมโดยตรง แทนการพึ่งนโยบาย endorsement ภายนอก ทั้งนี้การเปรียบเทียบนี้เป็นเชิงคุณภาพ เนื่องจากยังไม่มีกรวัดแบบ head-to-head บนภาระงานพลังงานเดียวกัน

6.2 ข้อจำกัด

ข้อจำกัดสำคัญของงานนี้คือระบบยังอยู่ในระดับการจำลอง โดยข้อมูลพลังงานมาจาก Smart Meter Simulator และยังไม่ได้ทดสอบร่วมกับอุปกรณ์ Smart Meter จริงหรือระบบไฟฟ้าภาคสนาม ดังนั้นผลลัพธ์จึงสะท้อนความเหมาะสมของสถาปัตยกรรมและกระบวนการทำงานเบื้องต้น มากกว่าประสิทธิภาพเชิงปริมาณในสภาพแวดล้อมจริง

นอกจากนี้การประเมินยังครอบคลุมเฉพาะบางส่วนของเส้นทางการทำงานทั้งหมด กล่าวคือวัดอัตราการรับข้อมูลของเส้นทาง telemetry ingest และต้นทุน compute ของคำสั่งชำระธุรกรรมบนเชนเป็นรายส่วน แต่ยังไม่ได้วัดความหน่วงแบบ end-to-end ตั้งแต่ค่าอ่านมาตรวัดจนถึงการยืนยันบนเชน และอัตราการจับคู่คำสั่งภายใต้ปริมาณคำสั่งหลายระดับ ส่วนปริมาณงานธุรกรรมของเส้นทางชำระวัดบน validator เดียวแล้ว แต่ยังไม่ครอบคลุมเครือข่าย permissioned หลายโหนดที่รวมต้นทุน consensus และยังไม่ประเมิน batch settlement อย่างเป็นทางการทดลองจริง ในด้านความเชื่อถือ เงื่อนไขปริมาณพลังงานและ Grid stability ถูกบังคับใช้ในชั้น off-chain โดย Aggregator Bridge และ oracle authority ซึ่งการสมรู้ร่วมคิดหรือการถูกประนีประนอมของส่วนนี้ยังไม่ถูกป้องกันด้วยกลไกเชิงรหัสวิทยา

6.3 ข้อควรระวังในการตีความผล

ผลการวัดในงานนี้มีข้อควรระวังในการตีความสามประการ ประการแรก (internal validity) ภาระงานในการทดลอง ingest ถูกสร้างจากไคลเอนต์ผู้ส่งเพียงตัวเดียวที่ทำงานแบบ asynchronous อัตราการรับข้อมูลที่วัดได้จึงเป็น lower bound ที่สะท้อนข้อจำกัดฝั่งผู้ส่งร่วมด้วย มิใช่เพดานความสามารถที่แท้จริงของบริการรับข้อมูล ประการที่สอง (construct validity) อัตราเชิงออกแบบ 5.33 รายการต่อวินาทีคำนวณจากจำนวนมาตรวัดและรอบการส่ง มิใช่อัตราสูงสุดที่วัดได้ การเปรียบเทียบตัวเลขทั้งสองจึงต้องระบุบริบทของแต่ละค่าให้ชัด ประการที่สาม (external validity) ต้นทุน compute ของคำสั่งชำระธุรกรรมวัดบน solana-test-validator รุ่นเดียว และค่าดังกล่าวเป็นต้นทุนเชิงตรรกะของโปรแกรมที่ไม่ขึ้นกับฮาร์ดแวร์ แต่ความหน่วงและปริมาณงานบนเครือข่าย production จริงอาจต่างออกไป

6.4 ทิศทางการประเมินเชิงปริมาณที่เปิดไว้

นอกเหนือจากข้างต้น ทิศทางการประเมินเชิงปริมาณที่เปิดไว้อย่างชัดเจนมีสามประการ ได้แก่

- **ประสิทธิภาพการจัดสรรเชิงสวัสดิการ (allocative efficiency / welfare):** วัดส่วนเกินจากการซื้อขาย (gains from trade) ของกลไก CDA เทียบกับฐานราคาเดียว (uniform-price clearing) และ feed-in tariff บนข้อมูลการจับคู่จริงจากภาระงานจำลองเต็มรูปแบบ เพื่อยืนยันส่วนเพิ่มเชิงเศรษฐศาสตร์ที่วิเคราะห์เชิงแบบจำลองไว้ในหัวข้อ 4.5 ด้วยการวัดจริง
- **ต้นทุนต่อการซื้อขายในหน่วยเงิน (cost per trade):** แปลงต้นทุน compute ต่อธุรกรรมเป็นค่าธรรมเนียมในหน่วย lamport และเงินบาทที่อัตราที่กำหนด แล้วรายงานสัดส่วนต้นทุนต่อมูลค่าธุรกรรม (fee-to-trade-value ratio) ซึ่งเป็นเกณฑ์ตัดสินสำคัญต่อการนำไปใช้จริง
- **ความพร้อมใช้งานและการเปรียบเทียบฐาน (liveness & baseline):** ประเมินความพร้อมใช้งานของเครือข่าย permissioned เมื่อมีโหนด validator ล้มเหลวบางส่วน (1-of-N down) และวัดเปรียบเทียบแบบ head-to-head เชิงปริมาณกับแพลตฟอร์ม permissioned อื่น โดยเฉพาะ Hyperledger Fabric บนภาระงานพลังงานเดียวกัน

7. สรุปผลการศึกษาและข้อเสนอแนะ

สรุปผลการศึกษา:

โครงการนี้นำเสนอการพัฒนากระบวนการซื้อขายพลังงานแสงอาทิตย์แบบ Peer-to-Peer สำหรับไม่โครกริด โดยผสมผสาน Smart Meter Simulator, Aggregator Bridge และ Solana-compatible Smart Contract เข้าด้วยกัน ระบบถูกออกแบบให้รองรับข้อมูลพลังงานแบบเวลาจริง ตรวจสอบคำสั่งซื้อขายผ่านบริการ Backend และบันทึกผลธุรกรรมที่ผ่านเงื่อนไขลงบนบล็อกเชนเพื่อเพิ่มความโปร่งใสและความสามารถในการตรวจสอบย้อนหลัง ผลการประเมินเชิงสถาปัตยกรรมชี้ให้เห็นว่าแนวทาง Microservice ร่วมกับการจัดคิวธุรกรรมผ่าน NATS JetStream มีศักยภาพในการลดการผูกติดกันของภาระงานระหว่างการรับข้อมูล การตรวจสอบธุรกรรม และการบันทึกผลบนบล็อกเชนได้ นอกจากนี้ การจำกัดบทบาทของ Smart Contract ให้เป็นชั้น Settlement and Audit layer ช่วยกำหนดขอบเขต compute บนบล็อกเชนให้ชัดเจนขึ้น การประเมินครอบคลุมสี่ด้าน ได้แก่ (1) เส้นทางรับข้อมูลที่รองรับอัตราเชิงออกแบบ 5.33 รายการต่อวินาทีโดยไม่มีการสูญหาย และคงสัดส่วนการสูญหายไว้ไม่เกิน 0.03% ถึงภาระ 640 เครื่อง (2) เครื่องจับคู่ CDA ที่ประมวลผลได้ประมาณ 3.1×10^4 คู่คำสั่งต่อวินาทีในหน่วยความจำ (3) ต้นทุนการชำระบนเชน 96,707 compute units ต่อคู่คำสั่ง หรือราว 48% ของงบ compute ตั้งต้น และ (4) ปริมาณงานการชำระจริงราว 0.5 ธุรกรรมต่อวินาทีบน validator เดียว ซึ่งถูกผูกกับการเขียนบัญชีส่วนกลางโดยการออกแบบ แต่ยังรองรับได้ราว 450 การชำระต่อหน้าต่งเคลียร์ตลาด 900 วินาที ผลทั้งสี่ด้านสนับสนุนการออกแบบที่ให้บล็อกเชนเป็นชั้น settlement แบบบาง โดยการที่ต้นทุนบนเชนเป็นค่าที่กำหนดได้แน่นอน มากกว่าจะผันแปรตามค่าธรรมเนียมตลาดแบบเครือข่ายสาธารณะ ยังสอดคล้องกับเหตุผลเชิงการกำกับดูแลในการเลือกเครือข่าย consortium แบบ PoA ส่วนการวัดความหน่วงแบบ end-to-end การทดลอง batch settlement และการประเมินบนอุปกรณ์และเครือข่ายจริงเป็นงานในลำดับถัดไป ซึ่งเป็นรากฐานสำคัญสำหรับการต่อยอดสู่การใช้งานในระดับอุตสาหกรรมพลังงานในอนาคต

ข้อเสนอแนะ:

ข้อจำกัดสำคัญของงานนี้คือระบบยังอยู่ในระดับการจำลอง โดยข้อมูลพลังงานมาจาก Smart Meter Simulator และยังไม่ได้ทดสอบร่วมกับอุปกรณ์ Smart Meter จริงหรือระบบไฟฟ้าภาคสนาม ผลลัพธ์จึงสะท้อนความเหมาะสมของสถาปัตยกรรมมาก

กว่าประสิทธิภาพเชิงปริมาณในสภาพแวดล้อมจริง นอกจากนี้อัตราการรับข้อมูลที่วัดได้เป็น lower bound ของไคลเอนต์ผู้ส่งเดียว และปริมาณงานการชำระวัดบน validator เดียวที่ยังไม่รวมต้นทุน consensus ของเครือข่าย permissioned หลายโหนด ในด้านความเชื่อถือ เงื่อนไขปริมาณพลังงานและ Grid stability ถูกบังคับใช้ในชั้น off-chain โดย Aggregator Bridge และ oracle authority ซึ่งการสมรู้ร่วมคิดหรือการถูกประนีประนอมของส่วนนี้ยังไม่ถูกป้องกันด้วยกลไกเชิงรหัสวิทยา

งานในอนาคตจึงควรพัฒนาการเชื่อมต่อกับ Smart Meter จริงผ่านมาตรฐาน DLMS/COSEM วัดความหน่วงแบบ end-to-end ขยายการวัด throughput ของ settlement ไปสู่เครือข่าย permissioned หลายโหนดที่รวมต้นทุน consensus ทดลอง batch settlement ด้วยวิธีบรรจูลายเส้นแบบใหม่ วัดเพดานของเส้นทางรับข้อมูลด้วยผู้ส่งแบบขนานหลายตัว และลด residual trust ของชั้น off-chain ด้วย multi-oracle attestation หรือ cryptographic proof

บรรณานุกรม

- [1] W. Tushar, T. K. Saha, C. Yuen, D. Smith, and H. V. Poor, "Peer-to-Peer Trading in Electricity Networks: An Overview," *IEEE Transactions on Smart Grid*, vol. 11, no. 4, pp. 3185–3200, 2020.
- [2] J. Guerrero, A. C. Chapman, and G. Verbic, "Decentralized P2P Energy Trading Under Network Constraints in a Low-Voltage Network," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 5163–5173, 2019.
- [3] A. Esmat, M. de Vos, Y. Ghiassi-Farrokhfal, P. Palensky, and D. Epema, "A Novel Decentralized Platform for Peer-to-Peer Energy Trading Market with Blockchain Technology," *Applied Energy*, vol. 282, p. 116123, 2021.
- [4] M. Andoni, V. Robu, D. Flynn, S. Abram, D. Geach, D. Jenkins, P. McCallum, and A. Peacock, "Blockchain Technology in the Energy Sector: A Systematic Review of Challenges and Opportunities," *Renewable and Sustainable Energy Reviews*, vol. 100, pp. 143–174, 2019.
- [5] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," White paper, 2008. <https://bitcoin.org/bitcoin.pdf>
- [6] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," White paper, 2014. <https://ethereum.org/en/whitepaper/>
- [7] D. Yaga, P. Mell, N. Roby, and K. Scarfone, "Blockchain Technology Overview," National Institute of Standards and Technology, NIST IR 8202, 2018.
- [8] A. Yakovenko, "Solana: A New Architecture for a High Performance Blockchain," White paper v0.8.13, 2018. <https://solana.com/solana-whitepaper.pdf>
- [9] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proc. 3rd Symposium on Operating Systems Design and Implementation (OSDI)*, 1999, pp. 173–186.
- [10] E. Mengelkamp, P. Staudt, J. Garttner, and C. Weinhardt, "Trading on Local Energy Markets: A Comparison of Market Designs and Bidding Strategies," *Applied Energy*, vol. 210, pp. 870–880, 2018.
- [11] E. Munsing, J. Mather, and S. Moura, "Blockchains for Decentralized Optimization of Energy Resources in Microgrid Networks," in *Proc. 2017 IEEE Conference on Control Technology and Applications*, pp. 2164–2171.
- [12] J. Kang, R. Yu, X. Huang, S. Maharjan, Y. Zhang, and E. Hossain, "Enabling Localized Peer-to-Peer Electricity Trading Among Plug-in Hybrid Electric Vehicles Using Consortium Blockchains," *IEEE Transactions on Industrial Informatics*, vol. 13, no. 6, pp. 3154–3164, 2017.
- [13] E. Androulaki et al., "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains," in *Proc. Thirteenth EuroSys Conference*, 2018, pp. 1–15.
- [14] NATS.io, "JetStream," Online documentation, 2024. <https://docs.nats.io/nats-concepts/jetstream>
- [15] Coral, "Anchor Documentation," Online documentation, 2026. <https://www.anchor-lang.com/docs>
- [16] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function Using NetworkX," in *Proc. 7th Python in Science Conference*, 2008, pp. 11–15.
- [17] K. S. Anderson, C. W. Hansen, W. F. Holmgren, A. R. Jensen, M. A. Mikofski, and A. Driesse, "pvlib python: 2023 Project Update," *Journal of Open Source Software*, vol. 8, no. 92, p. 5994, 2023.
- [18] L. Thurner, A. Scheidler, F. Schaefer, J.-H. Menke, J. Dollichon, F. Meier, S. Meinecke, and M. Braun, "pandapower: An Open-Source Python Tool for Convenient Modeling, Analysis, and Optimization of Electric Power Systems," *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018.
- [19] D. P. Chassin, K. Schneider, and C. Gerkensmeyer, "GridLAB-D: An Open-Source Power Systems Modeling and Simulation Environment," in *Proc. 2008 IEEE/PES Transmission and Distribution Conference and Exposition*, pp. 1–5.
- [20] T. Morstyn, A. Teytelboym, and M. D. McCulloch, "Bilateral Contract Networks for Peer-to-Peer Energy Trading," *IEEE Transactions on Smart Grid*, vol. 10, no. 2, pp. 2026–2035, 2019.
- [21] A. Paudel, K. Chaudhari, C. Long, and H. B. Gooi, "Peer-to-Peer Energy Trading in a Prosumer-Based Community Microgrid: A Game-Theoretic Model," *IEEE Transactions on Industrial Electronics*, vol. 66, no. 8, pp. 6087–6097, 2019.

-
- [22] IEEE Standards Association, "IEEE Std 1547-2018: IEEE Standard for Interconnection and Interoperability of Distributed Energy Resources with Associated Electric Power Systems Interfaces," IEEE Standard, 2018.
- [23] IEEE Standards Association, "IEEE Std 2030.7-2017: IEEE Standard for the Specification of Microgrid Controllers," IEEE Standard, 2018.
- [24] IEEE Standards Association, "IEEE Std 2030.8-2018: IEEE Standard for the Testing of Microgrid Controllers," IEEE Standard, 2018.
- [25] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Yellow paper, 2014. <https://ethereum.github.io/yellowpaper/paper.pdf>
- [26] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.
- [27] S. Joshi, "Feasibility of Proof of Authority as a Consensus Protocol Model," arXiv preprint arXiv:2109.02480, 2021.
- [28] Z. Tanis, A. Durusu, and N. Altintas, "A Comprehensive Review on Peer-to-Peer Energy Trading: Market Structure, Operational Layers, Energy Cooperatives and Multi-energy Systems," *IET Renewable Power Generation*, vol. 19, no. 1, 2025.
- [29] G. B. Bhavana *et al.*, "Applications of Blockchain Technology in Peer-to-Peer Energy Markets and Green Hydrogen Supply Chains: A Topical Review," *Scientific Reports*, vol. 14, no. 1, 2024.
- [30] G. B. Bhavana *et al.*, "Comparative Evaluation and Simulation of Blockchain Consensus Mechanisms for Secure and Scalable Peer-to-Peer Energy Trading in Microgrids," *Scientific Reports*, vol. 15, no. 1, 2025.
- [31] Solana Foundation, "Solana Documentation," Online documentation, 2026. <https://solana.com/docs>
- [32] Solana Foundation, "Program Derived Addresses," Online documentation, 2026. <https://solana.com/docs/core/pda>
- [33] Solana Foundation, "Tokens on Solana," Online documentation, 2026. <https://solana.com/docs/tokens>
- [34] SPIFFE Project, "X.509-SVID Specification," Online documentation, 2018. <https://spiffe.io/docs/latest/spiffe-specs/x509-svid/>
- [35] Python Software Foundation, "Python 3.11 Documentation," 2026. <https://docs.python.org/3.11/>
- [36] S. Ramirez, "FastAPI Documentation," 2026. <https://fastapi.tiangolo.com/>